

Geometry_MMALS_G1_FunctionalRouting_v1_1_0

June 14, 2026

1 Geometry-MMALS G1 v1.1.0

1.1 Functional Geometry-Mediated Routing and Host Allocation

Status: complete protocol implementation; not yet executed

Depends on: the replicated v1.0.9 five-seed context-geometry result

Primary question: can the ordered context representation be converted into ordered, causally specific, and functionally meaningful host allocation?

1.1.1 What the program builds, in concrete terms

Imagine training a system to recognize handwritten digits rotated to -60° , -30° , 0° , 30° , and 60° . The system sees these conditions progressively, one continual-learning stage at a time.

Inside the system:

- z_0 is the **frozen sensory view** of the image;
- $c \in \mathbb{R}^4$ is the **inferred context**, a four-number description of the current situation;
- $r \in \Delta^{H-1}$ is the **route**, a probability distribution assigning work to H hosts;
- g_h are the **hosts**, small specialist transformations;
- $z_M = \sum_h r_h g_h(z_0)$ is the **host synthesis** used for classification.

v1.0.9 established Level 1 evidence: the context space reflects rotation order across five model seeds and generalizes to source identities not used to fit the decoder. It did **not** establish Level 2: the route did not reproducibly inherit that order. v1.1.0 freezes the successful map and tests three mechanisms for reading it.

1.1.2 Three evidence levels

1. **Representational** — the context reflects the external factor. Achieved as replicated candidate evidence in v1.0.9.
2. **Functional** — ordered context changes cause ordered and meaningful changes in host allocation. This notebook tests that bridge.
3. **Operational** — the bridge improves accuracy, retention, robustness, or cost. Not assumed and not required for the primary v1.1.0 claim.

```
[4]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

1.2 0. Release-integrity setup

The notebook installs the repository in editable mode and verifies the exact scientific version. It does not patch installed source code at runtime. The canonical Python functions live under `src/geometry_mmalls/` and are covered by package tests.

```
[5]: import os, sys, shutil, importlib, subprocess
    from pathlib import Path

    EXPECTED_PACKAGE_VERSION = "1.1.0"
    EXPECTED_BUILD_REVISION = "functional-routing-bridge-c0-r1"
    REPO_URL = "https://github.com/gharbonnier78/geometry-mmalls-g1.git"
    REPO_DIR = Path("/content/geometry-mmalls-g1")
    SRC_DIR = REPO_DIR / "src"
    FORCE_REFRESH = False

    if FORCE_REFRESH and REPO_DIR.exists():
        shutil.rmtree(REPO_DIR)
    if not REPO_DIR.exists():
        subprocess.run(["git", "clone", REPO_URL, str(REPO_DIR)], check=True)

    os.chdir(REPO_DIR)
    subprocess.run(
        [sys.executable, "-m", "pip", "install", "--no-cache-dir", "--no-deps", "-e", str(REPO_DIR)],
        check=True,
    )
    if str(SRC_DIR) not in sys.path:
        sys.path.insert(0, str(SRC_DIR))
    for name in list(sys.modules):
        if name == "geometry_mmalls" or name.startswith("geometry_mmalls."):
            del sys.modules[name]
    importlib.invalidate_caches()
    import geometry_mmalls

    loaded_from = Path(geometry_mmalls.__file__).resolve()
    assert loaded_from.parent == (SRC_DIR / "geometry_mmalls").resolve()
    assert geometry_mmalls.__version__ == EXPECTED_PACKAGE_VERSION, (
        f"Expected {EXPECTED_PACKAGE_VERSION}, loaded {geometry_mmalls.__version__}.
    ↪ "
        "Push the v1.1.0 package and rerun with FORCE_REFRESH=True."
    )
    print("Geometry-MMALS", geometry_mmalls.__version__, "from", loaded_from)
```

Geometry-MMALS 1.1.0 from /content/geometry-mmalls-g1/src/geometry_mmalls/__init__.py

1.3 1. Configuration, profiles, and preregistered treatments

The primary comparison contains no angle-supervised route loss. All routers see the same frozen context, hosts start from the same seed-specific state, and the training budget is matched.

- **R0 MLP:** flexible context-only baseline;
- **R1 Linear:** tests whether a low-capacity map transmits context order;
- **R2 Prototype-energy:** assigns hosts according to distance from learned context prototypes.

The geometric energy of host h is

$$E_h^{\text{geo}}(c) = \frac{d_C(c, \mu_h)^2}{2\sigma_h^2} + b_h, \quad d_C(a, b) = \frac{1}{2}\|a - b\|_2,$$

and

$$r_h(c) = \frac{\exp[-E_h^{\text{geo}}(c)/\tau]}{\sum_k \exp[-E_k^{\text{geo}}(c)/\tau]}.$$

This is the first controlled component of the broader Energy-Guided Router; cost, memory, stability, host state, goals, and future control are deliberately excluded here.

```
[6]: from pathlib import Path
import copy, hashlib, json, math, random, time
from dataclasses import asdict

import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F
import yaml
from scipy import stats
from scipy.stats import spearmanr
from sklearn.linear_model import Ridge
from sklearn.metrics import r2_score
from torch import nn
from torch.utils.data import DataLoader, Subset
from torchvision.datasets import MNIST

from geometry_mmalls.data import FixedAngleMNIST, MultiAngleMNIST
from geometry_mmalls.geometry import (
    paired_route_geometry_loss_stationary,
    paired_context_geometry_loss,
    context_path_spread_loss,
    cross_source_fiber_alignment_loss,
    factor_centroid_geometry_loss,
    normalized_stress,
)
from geometry_mmalls.metrics import (
```

```

        bootstrap_mean_ci,
        centroid_geometry_scores,
        grouped_geometry_scores,
        ridge_factor_probe,
    )
from geometry_mmalls.model import GeometryMMALS, ResidualHost, SmallConvEncoder
from geometry_mmalls.functional_routing import (
    FunctionalRoutingMMALS,
    LinearContextRouter,
    MLPContextRouter,
    PrototypeEnergyRouter,
    entropic_transport_cost,
    host_functional_diversity_loss,
    host_output_cost_matrix,
    host_specialization,
    host_territory,
    pairwise_functional_route_distances,
    parameter_count,
    reconstruct_route_from_root_probe,
    root_simplex_chord,
    territory_overlap,
    usage_balance_loss,
)

NOTEBOOK_VERSION = "1.1.0"
BUILD_REVISION = "functional-routing-bridge-c0-r1"
base_config = yaml.safe_load(Path("configs/rotated_mnist_g1.yaml").read_text())
protocol = yaml.safe_load(Path("configs/rotated_mnist_g1_v110.yaml").
    ↪read_text())
assert protocol["build_revision"] == BUILD_REVISION

# development: one seed; pilot: five seeds; qualification: ten seeds.
RUN_PROFILE = os.environ.get("G1_PROFILE", "pilot")
RUN_SECONDARY_MEDIATION_ARM = False
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
SPLIT_SEED = int(protocol["experiment"]["fixed_split_seed"])
SENSORY_SEED = int(protocol["experiment"]["fixed_sensory_seed"])

if RUN_PROFILE == "development":
    MODEL_SEEDS = [0]
    TRAIN_SOURCE_LIMIT, TEST_SOURCE_LIMIT = 256, 320
    SENSORY_SOURCE_LIMIT, SENSORY_TEST_LIMIT = 1500, 750
    SENSORY_EPOCHS, CONTEXT_STAGE_EPOCHS, ROUTER_STAGE_EPOCHS = 1, 1, 1
    SOURCE_BATCH_SIZE, SOURCE_BOOTSTRAP_SAMPLES = 64, 300
    SENSORY_GATE = 0.80
elif RUN_PROFILE == "pilot":
    MODEL_SEEDS = list(map(int, protocol["experiment"]["pilot_model_seeds"]))

```

```

TRAIN_SOURCE_LIMIT, TEST_SOURCE_LIMIT = 512, 640
SENSORY_SOURCE_LIMIT, SENSORY_TEST_LIMIT = 2000, 1000
SENSORY_EPOCHS, CONTEXT_STAGE_EPOCHS, ROUTER_STAGE_EPOCHS = 2, 2, 2
SOURCE_BATCH_SIZE = 64
SOURCE_BOOTSTRAP_SAMPLES =  $\square$ 
 $\hookrightarrow$ int(protocol["statistics"]["source_bootstrap_samples"])
SENSORY_GATE = 0.85
elif RUN_PROFILE == "qualification":
    MODEL_SEEDS = list(map(int,  $\square$ 
 $\hookrightarrow$ protocol["experiment"]["qualification_model_seeds"])))
    TRAIN_SOURCE_LIMIT =  $\square$ 
 $\hookrightarrow$ int(base_config["paired_protocol"]["full_source_limit"])
    TEST_SOURCE_LIMIT, SENSORY_SOURCE_LIMIT, SENSORY_TEST_LIMIT = 2500, 12000,  $\square$ 
 $\hookrightarrow$ 5000
    SENSORY_EPOCHS = int(base_config["training"]["sensory_pretrain_epochs"])
    CONTEXT_STAGE_EPOCHS = ROUTER_STAGE_EPOCHS =  $\square$ 
 $\hookrightarrow$ int(base_config["training"]["stage_epochs"])
    SOURCE_BATCH_SIZE = int(base_config["paired_protocol"]["source_batch_size"])
    SOURCE_BOOTSTRAP_SAMPLES =  $\square$ 
 $\hookrightarrow$ int(protocol["statistics"]["source_bootstrap_samples"])
    SENSORY_GATE = 0.95
else:
    raise ValueError("G1_PROFILE must be development, pilot, or qualification")

TRAIN_ANGLES = list(map(float, base_config["data"]["train_angles"]))
CURRICULUM = list(map(float, protocol["experiment"]["curriculum"]))
DENSE_EVAL_ANGLES = list(map(float,  $\square$ 
 $\hookrightarrow$ protocol["experiment"]["dense_eval_angles"])))
HELDOUT_ANGLES = [angle for angle in DENSE_EVAL_ANGLES if angle not in  $\square$ 
 $\hookrightarrow$ TRAIN_ANGLES]
CONTEXT_DIM = int(protocol["frozen_representation"]["context_dimension"])
NUM_HOSTS = int(protocol["experiment"]["host_count"])
TAU = float(protocol["training"]["temperature"])
METHOD_SPECS = protocol["routers"]
METHODS = list(METHOD_SPECS)
PRIMARY_CONTRASTS = protocol["primary_contrasts"]

print({
    "version": NOTEBOOK_VERSION,
    "revision": BUILD_REVISION,
    "profile": RUN_PROFILE,
    "device": str(DEVICE),
    "seeds": MODEL_SEEDS,
    "methods": METHODS,
    "secondary_arm": RUN_SECONDARY_MEDIATION_ARM,
})

```

```
{'version': '1.1.0', 'revision': 'functional-routing-bridge-c0-r1', 'profile':
'pilot', 'device': 'cpu', 'seeds': [0, 1, 2, 3, 4], 'methods': ['r0_mlp',
'r1_linear', 'r2_prototype_energy'], 'secondary_arm': False}
```

1.4 2. Numerical and architectural self-tests

These gates check probability normalization, prototype projection, finite gradients for coincident contexts, and finite entropic transport before any expensive experiment starts.

```
[7]: torch.manual_seed(110)
probe_context = F.normalize(torch.randn(16, CONTEXT_DIM), dim=-1)
for router in [
    MLPContextRouter(CONTEXT_DIM, NUM_HOSTS),
    LinearContextRouter(CONTEXT_DIM, NUM_HOSTS),
    PrototypeEnergyRouter(CONTEXT_DIM, NUM_HOSTS),
]:
    route = router(probe_context)
    assert route.shape == (16, NUM_HOSTS)
    assert torch.isfinite(route).all()
    assert torch.allclose(route.sum(-1), torch.ones(16), atol=1e-6)

prototype = PrototypeEnergyRouter(CONTEXT_DIM, NUM_HOSTS)
prototype.initialize_from_contexts(probe_context, seed=0)
prototype.project_prototypes()
assert torch.allclose(prototype.prototypes.norm(dim=-1), torch.ones(NUM_HOSTS),
    atol=1e-6)

identical = torch.full((8, NUM_HOSTS), 1.0 / NUM_HOSTS, requires_grad=True)
root_loss = root_simplex_chord(identical, identical).mean()
root_loss.backward()
assert identical.grad is not None and torch.isfinite(identical.grad).all()

host_probe = torch.randn(32, NUM_HOSTS, 24)
cost_probe = host_output_cost_matrix(host_probe)
route_a = torch.softmax(torch.randn(32, NUM_HOSTS), dim=-1)
route_b = torch.softmax(torch.randn(32, NUM_HOSTS), dim=-1)
ot_probe = entropic_transport_cost(route_a, route_b, cost_probe, epsilon=0.05)
assert torch.isfinite(ot_probe).all()

print("Router, prototype, root-chord, and Sinkhorn gates: PASS")
```

Router, prototype, root-chord, and Sinkhorn gates: PASS

1.5 3. Source-disjoint partitions and frozen sensory grove

Five disjoint test-source partitions are used for distinct measurements:

1. host-function cost construction;
2. mediation-probe fitting;

3. mediation-probe evaluation;
4. tangent fitting;
5. final causal and ablation evaluation.

The sensory encoder is trained once at 0° and then frozen for all seeds and treatments.

```
[8]: root = Path(base_config["data"]["root"])
base_train = MNIST(root=str(root), train=True, download=True)
base_test = MNIST(root=str(root), train=False, download=True)

split_rng = np.random.default_rng(SPLIT_SEED)
train_perm = split_rng.permutation(len(base_train))
test_perm = split_rng.permutation(len(base_test))
sensory_indices = train_perm[:SENSORY_SOURCE_LIMIT].tolist()
train_indices = train_perm[:TRAIN_SOURCE_LIMIT].tolist()
test_indices = test_perm[:TEST_SOURCE_LIMIT].tolist()
sensory_test_indices = test_perm[:SENSORY_TEST_LIMIT].tolist()

partition_arrays = np.array_split(np.asarray(test_indices), 5)
metric_indices = partition_arrays[0].tolist()
probe_fit_indices = partition_arrays[1].tolist()
probe_eval_indices = partition_arrays[2].tolist()
tangent_fit_indices = partition_arrays[3].tolist()
causal_eval_indices = partition_arrays[4].tolist()

def checksum(values):
    return hashlib.sha256(",".join(map(str, values)).encode()).hexdigest()

def generator(seed):
    g = torch.Generator(); g.manual_seed(int(seed)); return g

def fixed_loader(angle, train, indices, shuffle, loader_seed=0, batch_size=128):
    ds = Subset(FixedAngleMNIST(root, angle=angle, train=train, download=True),
    ↪indices)
    return DataLoader(
        ds, batch_size=batch_size, shuffle=shuffle, num_workers=0,
        generator=generator(loader_seed) if shuffle else None,
    )

def multi_loader(angles, train, indices, shuffle, loader_seed=0,
    ↪batch_size=None):
    ds = MultiAngleMNIST(root, angles=angles, train=train, indices=indices,
    ↪download=True)
    return DataLoader(
```

```

        ds, batch_size=batch_size or SOURCE_BATCH_SIZE, shuffle=shuffle,
        ↪num_workers=0,
        generator=generator(loader_seed) if shuffle else None,
    )

split_manifest = {
    "split_seed": SPLIT_SEED,
    "train_sha256": checksum(train_indices),
    "test_sha256": checksum(test_indices),
    "metric_sha256": checksum(metric_indices),
    "probe_fit_sha256": checksum(probe_fit_indices),
    "probe_eval_sha256": checksum(probe_eval_indices),
    "tangent_fit_sha256": checksum(tangent_fit_indices),
    "causal_eval_sha256": checksum(causal_eval_indices),
}

random.seed(SENSORY_SEED); np.random.seed(SENSORY_SEED); torch.
    ↪manual_seed(SENSORY_SEED)
if torch.cuda.is_available(): torch.cuda.manual_seed_all(SENSORY_SEED)
latent_dim = int(base_config["model"]["latent_dim"])
encoder = SmallConvEncoder(latent_dim).to(DEVICE)
sensory_head = nn.Linear(latent_dim, 10).to(DEVICE)
sensory_optimizer = torch.optim.AdamW(
    list(encoder.parameters()) + list(sensory_head.parameters()),
    lr=float(base_config["training"]["learning_rate"]),
    weight_decay=float(base_config["training"]["weight_decay"]),
)
sensory_history = []
for epoch in range(SENSORY_EPOCHS):
    total = correct = 0; loss_sum = 0.0
    encoder.train(); sensory_head.train()
    for x, y, _, _ in fixed_loader(0.0, True, sensory_indices, True,
    ↪SENSORY_SEED * 1000 + epoch):
        x, y = x.to(DEVICE), y.to(DEVICE)
        logits = sensory_head(encoder(x)); loss = F.cross_entropy(logits, y)
        sensory_optimizer.zero_grad(set_to_none=True); loss.backward()
        torch.nn.utils.clip_grad_norm_(list(encoder.parameters()) +
    ↪list(sensory_head.parameters()), 5.0)
        sensory_optimizer.step()
        total += y.numel(); correct += (logits.argmax(1) == y).sum().item()
        loss_sum += float(loss.detach()) * y.numel()
    sensory_history.append({"epoch": epoch, "loss": loss_sum / total,
    ↪"train_accuracy": correct / total})
    print(sensory_history[-1])

@torch.no_grad()
def evaluate_sensory():

```



```

    encoder.eval(); sensory_head.eval(); total = correct = 0
    for x, y, _, _ in fixed_loader(0.0, False, sensory_test_indices, False,
    ↪batch_size=256):
        x, y = x.to(DEVICE), y.to(DEVICE)
        correct += (sensory_head(encoder(x)).argmax(1) == y).sum().item();
    ↪total += y.numel()
    return correct / total

sensory_accuracy = evaluate_sensory()
assert RUN_PROFILE != "qualification" or sensory_accuracy >= SENSORY_GATE
sensory_state = copy.deepcopy(encoder.state_dict())
print("Sensory gate:", sensory_accuracy, "threshold:", SENSORY_GATE)

```

```

100%|      | 9.91M/9.91M [00:00<00:00, 57.1MB/s]
100%|      | 28.9k/28.9k [00:00<00:00, 3.42MB/s]
100%|      | 1.65M/1.65M [00:00<00:00, 14.2MB/s]
100%|      | 4.54k/4.54k [00:00<00:00, 2.39MB/s]

```

```

{'epoch': 0, 'loss': 1.5406257815361022, 'train_accuracy': 0.606}
{'epoch': 1, 'loss': 0.5623613724708557, 'train_accuracy': 0.909}
Sensory gate: 0.942 threshold: 0.85

```

1.6 4. Reproduce and freeze the v1.0.9 aligned context checkpoint

For each model seed, this stage reproduces the successful four-dimensional aligned context treatment using the v1.0.9 stationary objective. Only the sensory encoder and context encoder are transferred to v1.1.0. The context encoder is then frozen, so all router treatments receive exactly the same map.

This stage is separated from the routing phase to prevent the context and route from moving together and hiding causality.

```

[9]: def state_hash(state_dict):
    h = hashlib.sha256()
    for name in sorted(state_dict):
        tensor = state_dict[name].detach().cpu().contiguous()
        h.update(name.encode()); h.update(tensor.numpy().tobytes())
    return h.hexdigest()

def epoch_seed(model_seed, stage, epoch, offset=0):
    return int(model_seed) * 1_000_000 + stage * 10_000 + epoch + int(offset)

def build_context_source_model():
    enc = SmallConvEncoder(latent_dim).to(DEVICE); enc.
    ↪load_state_dict(sensory_state)
    return GeometryMMALS(

```

```

        enc, latent_dim=latent_dim, context_dim=CONTEXT_DIM,
        num_hosts=NUM_HOSTS,
        host_hidden_dim=int(base_config["model"]["host_hidden_dim"]),
        freeze_encoder=True, bottleneck_hidden_dim=64,
    ).to(DEVICE)

def context_source_forward(model, flat):
    z0 = model.encode(flat)
    raw, context = model.infer_context(z0)
    route = torch.softmax(model.context_bottleneck_router(context) / TAU,
        dim=-1)
    return model.synthesize(z0, context, route, context_raw=raw)

def train_aligned_context_checkpoint(model_seed):
    torch.manual_seed(model_seed + 1090)
    model = build_context_source_model()
    params = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.AdamW(
        params,
        lr=float(base_config["training"]["learning_rate"]),
        weight_decay=float(base_config["training"]["weight_decay"]),
    )
    rows = []
    for stage, current_angle in enumerate(CURRICULUM):
        seen = CURRICULUM[:stage + 1]
        totals = {key: 0.0 for key in [
            "loss", "current_ce", "anchor_ce", "route_geo", "context_geo",
            "context_spread", "fiber", "centroid", "host_diversity",
        ]}
        totals.update({"forward_images": 0, "optimizer_steps": 0,
            "source_examples": 0})
        model.train(); start = time.perf_counter()
        for epoch in range(CONTEXT_STAGE_EPOCHS):
            loader = multi_loader(seen, True, train_indices, True,
                epoch_seed(model_seed, stage, epoch, 109))
            for views, y, factors, _ in loader:
                batch, angle_count = views.shape[:2]
                flat = views.reshape(-1, *views.shape[2:]).to(DEVICE)
                y, factors = y.to(DEVICE), factors.to(DEVICE)
                trace = context_source_forward(model, flat)
                logits = trace.logits.reshape(batch, angle_count, -1)
                routes = trace.route.reshape(batch, angle_count, -1)
                contexts = trace.context.reshape(batch, angle_count, -1)
                hosts = trace.host_outputs.reshape(batch, angle_count,
                    NUM_HOSTS, -1)

```

```

current_ce = F.cross_entropy(logits[:, -1], y)
anchor_ce = (
    F.cross_entropy(
        logits[:, :-1].reshape(-1, logits.shape[-1]),
        y[:, None].expand(-1, angle_count - 1).reshape(-1),
    ) if angle_count > 1 else logits.sum() * 0.0
)
route_geo = paired_route_geometry_loss_stationary(routes,
↪factors)

context_geo = paired_context_geometry_loss(contexts, factors)
spread = context_path_spread_loss(contexts, 0.05)
fiber = cross_source_fiber_alignment_loss(contexts, factors)
centroid = factor_centroid_geometry_loss(contexts, factors)
diversity = host_functional_diversity_loss(hosts[:, -1])
loss = (
    current_ce + 0.10 * anchor_ce + 0.10 * route_geo
    + 0.10 * context_geo + 0.05 * spread + 0.05 * fiber
    + 0.05 * centroid + 0.05 * diversity
)
components = torch.stack([loss, current_ce, anchor_ce,
↪route_geo, context_geo, spread, fiber, centroid, diversity])
if not torch.isfinite(components).all():
    raise FloatingPointError(f"Non-finite context-source loss,
↪seed={model_seed}, stage={stage}")
optimizer.zero_grad(set_to_none=True); loss.backward()
grad = torch.nn.utils.clip_grad_norm_(params, 5.0)
if not torch.isfinite(torch.as_tensor(grad)):
    raise FloatingPointError("Non-finite context-source
↪gradient")
optimizer.step()
totals["source_examples"] += batch; totals["forward_images"] +=
↪batch * angle_count
totals["optimizer_steps"] += 1
for key, value in {
    "loss": loss, "current_ce": current_ce, "anchor_ce":
↪anchor_ce,
    "route_geo": route_geo, "context_geo": context_geo,
    "context_spread": spread, "fiber": fiber, "centroid":
↪centroid,
    "host_diversity": diversity,
}.items():
    totals[key] += float(value.detach()) * batch
count = max(totals["source_examples"], 1)
row = {
    "model_seed": model_seed, "stage": stage, "current_angle":
↪current_angle,

```

```

        "seen_angles": json.dumps(seen), "wall_seconds": time.
↪perf_counter() - start,
        "forward_images": totals["forward_images"], "optimizer_steps":
↪totals["optimizer_steps"],
        **{key: totals[key] / count for key in totals if key not in
↪{"forward_images", "optimizer_steps", "source_examples"}},
    }
    rows.append(row); print("context", row)
    checkpoint = {
        "encoder": copy.deepcopy(model.encoder.state_dict()),
        "context_net": copy.deepcopy(model.context_net.state_dict()),
    }
    return model, checkpoint, state_hash(checkpoint["context_net"]), pd.
↪DataFrame(rows)

@torch.no_grad()
def collect_context_bank(source_model, angles, indices):
    contexts = []
    source_model.eval()
    for views, _, _, _ in multi_loader(angles, True, indices, False):
        flat = views.reshape(-1, *views.shape[2:]).to(DEVICE)
        z0 = source_model.encode(flat)
        _, context = source_model.infer_context(z0)
        contexts.append(context.cpu())
    return torch.cat(contexts, dim=0)

```

1.7 5. Build matched routing treatments

Hosts, synthesis normalization, and classifier start from one common seed-specific state. Router architectures differ by design, and their parameter counts are reported rather than hidden.

The routing-phase loss is

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_A \mathcal{L}_{\text{anchor}} + \lambda_B \mathcal{L}_{\text{balance}} + \lambda_D \mathcal{L}_{\text{host-div.}}$$

No angle-supervised route geometry appears in the primary comparison.

```

[10]: def build_router(method, context_bank, model_seed):
    spec = METHOD_SPECS[method]
    kind = spec["type"]
    torch.manual_seed(model_seed + int(spec["seed_offset"]))
    if kind == "mlp_context_router":
        router = MLPContextRouter(CONTEXT_DIM, NUM_HOSTS,
↪hidden_dim=int(spec["hidden_dim"]))
        init_diag = {}
    elif kind == "linear_context_router":
        router = LinearContextRouter(CONTEXT_DIM, NUM_HOSTS)

```

```

        init_diag = {}
    elif kind == "prototype_energy_router":
        router = PrototypeEnergyRouter(
            CONTEXT_DIM, NUM_HOSTS,
            sigma_min=float(spec["sigma_min"]), temperature=TAU,
        )
        init_diag = router.initialize_from_contexts(
            context_bank.to(DEVICE), seed=model_seed + int(spec["seed_offset"]),
            iterations=int(spec["spherical_kmeans_iterations"]),
        )
    else:
        raise ValueError(f"Unknown router type: {kind}")
    return router.to(DEVICE), init_diag

def create_common_host_state(model_seed):
    torch.manual_seed(model_seed + 1100)
    hosts = nn.ModuleList([
        ResidualHost(latent_dim, int(base_config["model"]["host_hidden_dim"]))
        for _ in range(NUM_HOSTS)
    ])
    synthesis_norm = nn.LayerNorm(latent_dim)
    classifier = nn.Linear(latent_dim, 10)
    return {
        "hosts": copy.deepcopy(hosts.state_dict()),
        "synthesis_norm": copy.deepcopy(synthesis_norm.state_dict()),
        "classifier": copy.deepcopy(classifier.state_dict()),
    }

def build_routing_model(method, checkpoint, common_state, context_bank,
    ↪model_seed):
    enc = SmallConvEncoder(latent_dim); enc.
    ↪load_state_dict(checkpoint["encoder"])
    context_net = nn.Sequential(nn.Linear(latent_dim, 64), nn.GELU(), nn.
    ↪Linear(64, CONTEXT_DIM))
    context_net.load_state_dict(checkpoint["context_net"])
    hosts = [ResidualHost(latent_dim,
    ↪int(base_config["model"]["host_hidden_dim"])) for _ in range(NUM_HOSTS)]
    host_list = nn.ModuleList(hosts); host_list.
    ↪load_state_dict(common_state["hosts"])
    synthesis_norm = nn.LayerNorm(latent_dim); synthesis_norm.
    ↪load_state_dict(common_state["synthesis_norm"])
    classifier = nn.Linear(latent_dim, 10); classifier.
    ↪load_state_dict(common_state["classifier"])
    router, init_diag = build_router(method, context_bank, model_seed)

```

```

model = FunctionalRoutingMMALS(
    enc, context_net, router, hosts, synthesis_norm, classifier,
    freeze_encoder=True, freeze_context=True,
).to(DEVICE)
return model, init_diag

def evaluate_routing_model(model, method, model_seed, stage, angles):
    rows = []; model.eval()
    with torch.no_grad():
        for angle in angles:
            total = correct = 0; nll_sum = 0.0
            for x, y, _, _ in fixed_loader(angle, False, test_indices, False,
↪batch_size=256):
                x, y = x.to(DEVICE), y.to(DEVICE)
                trace = model(x, temperature=TAU)
                total += y.numel(); correct += (trace.logits.argmax(1) == y).
↪sum().item()
                nll_sum += float(F.cross_entropy(trace.logits, y,
↪reduction="sum"))
            rows.append({
                "model_seed": model_seed, "method": method, "stage": stage,
                "angle": float(angle), "angle_type": "train" if angle in
↪TRAIN_ANGLES else "heldout",
                "accuracy": correct / total, "nll": nll_sum / total,
            })
    return rows

def train_router_treatment(model_seed, method, model, checkpoint_hash):
    params = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.AdamW(
        params,
        lr=float(base_config["training"]["learning_rate"]),
        weight_decay=float(base_config["training"]["weight_decay"]),
    )
    stage_rows = []; evaluation_rows = []
    for stage, current_angle in enumerate(CURRICULUM):
        seen = CURRICULUM[:stage + 1]
        totals = {key: 0.0 for key in ["loss", "current_ce", "anchor_ce",
↪"balance", "host_diversity"]}
        totals.update({"source_examples": 0, "forward_images": 0,
↪"optimizer_steps": 0})
        start = time.perf_counter(); model.train()
        for epoch in range(ROUTER_STAGE_EPOCHS):

```

```

        loader = multi_loader(seen, True, train_indices, True,
↪epoch_seed(model_seed, stage, epoch, 110))
        for views, y, _, _ in loader:
            batch, angle_count = views.shape[:2]
            flat = views.reshape(-1, *views.shape[2:]).to(DEVICE)
            y = y.to(DEVICE)
            trace = model(flat, temperature=TAU)
            logits = trace.logits.reshape(batch, angle_count, -1)
            routes = trace.route.reshape(batch, angle_count, -1)
            hosts = trace.host_outputs.reshape(batch, angle_count,
↪NUM_HOSTS, -1)
            current_ce = F.cross_entropy(logits[:, -1], y)
            anchor_ce = (
                F.cross_entropy(
                    logits[:, :-1].reshape(-1, logits.shape[-1]),
                    y[:, None].expand(-1, angle_count - 1).reshape(-1),
                ) if angle_count > 1 else logits.sum() * 0.0
            )
            balance = usage_balance_loss(routes)
            diversity = host_functional_diversity_loss(hosts[:, -1])
            loss = (
                current_ce
                + float(protocol["common_loss"]["retention_anchor"]) *
↪anchor_ce
                + float(protocol["common_loss"]["usage_balance"]) * balance
                + float(protocol["common_loss"]["host_diversity"]) *
↪diversity
            )
            components = torch.stack([loss, current_ce, anchor_ce, balance,
↪diversity])
            if not torch.isfinite(components).all():
                raise FloatingPointError(f"Non-finite route loss:")
↪seed={model_seed}, method={method}, stage={stage})
            optimizer.zero_grad(set_to_none=True); loss.backward()
            grad = torch.nn.utils.clip_grad_norm_(params, 5.0)
            if not torch.isfinite(torch.as_tensor(grad)):
                raise FloatingPointError("Non-finite routing gradient")
            optimizer.step()
            if isinstance(model.router, PrototypeEnergyRouter):
                model.router.project_prototypes_()
                totals["source_examples"] += batch; totals["forward_images"] +=
↪batch * angle_count
                totals["optimizer_steps"] += 1
                for key, value in {"loss": loss, "current_ce": current_ce,
↪"anchor_ce": anchor_ce, "balance": balance, "host_diversity": diversity}.
↪items():

```

```

        totals[key] += float(value.detach()) * batch
count = max(totals["source_examples"], 1)
row = {
    "model_seed": model_seed, "method": method, "stage": stage,
    "current_angle": current_angle, "seen_angles": json.dumps(seen),
    "context_checkpoint_hash": checkpoint_hash,
    "router_parameters": parameter_count(model.router),
    "forward_images": totals["forward_images"], "optimizer_steps":
↪totals["optimizer_steps"],
    "wall_seconds": time.perf_counter() - start,
    **{key: totals[key] / count for key in ["loss", "current_ce",
↪"anchor_ce", "balance", "host_diversity"]},
}
stage_rows.append(row); print(row)
evaluation_rows.extend(evaluate_routing_model(model, method,
↪model_seed, stage, DENSE_EVAL_ANGLES))
return model, pd.DataFrame(stage_rows), pd.DataFrame(evaluation_rows)

```

1.8 6. Trace collection and context-preservation audit

A frozen context should be numerically identical across R0, R1, and R2. The notebook records the maximum absolute context difference and the checkpoint hash for every treatment.

```

[11]: @torch.no_grad()
def collect_trace(model, method, model_seed, angles, indices):
    rows = []; model.eval()
    for views, labels, factors, source_ids in multi_loader(angles, False,
↪indices, False):
        batch, angle_count = views.shape[:2]
        trace = model(views.reshape(-1, *views.shape[2:]).to(DEVICE),
↪temperature=TAU)
        logits = trace.logits.reshape(batch, angle_count, -1)
        arrays = {
            "context": trace.context.reshape(batch, angle_count, -1).cpu().
↪numpy(),
            "route": trace.route.reshape(batch, angle_count, -1).cpu().numpy(),
            "z_mm": trace.z_mm.reshape(batch, angle_count, -1).cpu().numpy(),
            "logits": logits.cpu().numpy(),
        }
        log_probs = F.log_softmax(logits, dim=-1).cpu().numpy()
        for source in range(batch):
            label = int(labels[source])
            for angle_index in range(angle_count):
                rows.append({
                    "model_seed": model_seed, "method": method,
                    "source_index": int(source_ids[source]), "label": label,
                    "angle": float(factors[source, angle_index]),

```



```

        "correct": int(arrays["logits"][source, angle_index].
↪argmax() == label),
        "true_log_prob": float(log_probs[source, angle_index,
↪label]),
        "context": arrays["context"][source, angle_index],
        "route": arrays["route"][source, angle_index],
        "z_mm": arrays["z_mm"][source, angle_index],
    })
    return pd.DataFrame(rows)

def stack(frame, column):
    return np.stack(frame[column].to_numpy())

def context_preservation_table(trace):
    reference = trace[trace.method == METHODS[0]][["source_index", "angle",
↪"context"]].copy()
    rows = []
    for method in METHODS[1:]:
        other = trace[trace.method == method][["source_index", "angle",
↪"context"]]
        merged = reference.merge(other, on=["source_index", "angle"],
↪suffixes=("_ref", "_other"))
        delta = np.stack(merged.context_ref) - np.stack(merged.context_other)
        rows.append({
            "method": method,
            "reference": METHODS[0],
            "max_abs_context_delta": float(np.max(np.abs(delta))),
            "mean_l2_context_delta": float(np.linalg.norm(delta, axis=1).
↪mean()),
        })
    return pd.DataFrame(rows)

```

1.9 7. Nominal and functional route geometry

Historical route geometry treats hosts as fixed simplex coordinates:

$$d_R(r_a, r_b) = \frac{\|\sqrt{r_a} - \sqrt{r_b}\|_2}{\sqrt{2}}.$$

That metric cannot know whether two hosts perform nearly the same function. v1.1.0 therefore constructs a host-function cost matrix

$$C_{hk} = \mathbb{E}_x \left[\frac{\|g_h(z_0(x)) - g_k(z_0(x))\|_2^2}{d_M} \right]$$

on a fixed held-out source partition and measures route displacement by entropic optimal transport. The functional metric is evaluation-only in the primary experiment.

```
[12]: @torch.no_grad()
def build_host_cost(model, angles, indices):
    model.eval(); running = None; count = 0
    for views, _, _ in multi_loader(angles, False, indices, False):
        trace = model(views.reshape(-1, *views.shape[2:]).to(DEVICE),
        ↪temperature=TAU)
        outputs = trace.host_outputs
        delta = outputs[:, :, None, :] - outputs[:, None, :, :]
        batch_cost = torch.square(delta).sum(-1).sum(0) / float(outputs.
        ↪shape[-1])
        running = batch_cost if running is None else running + batch_cost
        count += outputs.shape[0]
    cost = running / max(count, 1)
    cost = 0.5 * (cost + cost.T); cost.fill_diagonal_(0.0)
    nonzero = cost[cost > 1e-12]
    if nonzero.numel(): cost = cost / nonzero.median()
    return cost.detach().cpu()

def pairwise_factor_distance(factors):
    values = np.asarray(factors, dtype=float)
    return np.abs(values[:, None] - values[None, :])

def pairwise_root_chord_np(routes):
    routes = np.asarray(routes, dtype=float)
    roots = np.sqrt(np.clip(routes, 1e-12, None))
    roots /= np.linalg.norm(roots, axis=1, keepdims=True)
    delta = roots[:, None, :] - roots[None, :, :]
    return np.linalg.norm(delta, axis=-1) / math.sqrt(2.0)

def geometry_score_from_matrices(reference, observed):
    mask = np.triu(np.ones_like(reference, dtype=bool), k=1)
    rho = spearmanr(reference[mask], observed[mask]).statistic
    return float(np.nan_to_num(rho)), normalized_stress(reference, observed,
    ↪mask=mask)

def grouped_route_geometry(frame, cost=None):
    rows = []
    for source_id, group in frame.groupby("source_index"):
        group = group.sort_values("angle")
        factors = group.angle.to_numpy(float)
```

```

    routes = stack(group, "route")
    reference = pairwise_factor_distance(factors)
    if cost is None:
        observed = pairwise_root_chord_np(routes)
        metric = "nominal_root_chord"
    else:
        observed = pairwise_functional_route_distances(
            torch.tensor(routes, dtype=torch.float32),
            cost.float(),
            ↵
↪epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
            ↵
↪iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
                ).numpy()
        metric = "functional_sinkhorn"
        rho, stress = geometry_score_from_matrices(reference, observed)
        rows.append({"source_index": source_id, "metric": metric, "rho": rho, ↵
↪"stress": stress})
    return pd.DataFrame(rows)

def bootstrap_source_mean(values, seed):
    values = np.asarray(values, dtype=float)
    rng = np.random.default_rng(int(seed) % (2 ** 32))
    draws = np.asarray([
        values[rng.integers(0, len(values), len(values))].mean()
        for _ in range(SOURCE_BOOTSTRAP_SAMPLES)
    ])
    return float(values.mean()), float(np.quantile(draws, 0.025)), float(np.
↪quantile(draws, 0.975))

```

1.10 8. Low-capacity mediation probes and controls

The probe asks whether a simple map from context to root-route coordinates generalizes to held-out identities and unseen angles. High training reconstruction is not enough.

Controls remove or scramble the factor-relevant context component:

- shuffled root-route targets;
- context with the fitted tangent removed;
- a matched one-dimensional orthogonal component.

```

[13]: def fit_context_tangent(frame):
    X = stack(frame, "context").astype(float)
    y = frame.angle.to_numpy(float)
    X0 = X - X.mean(0); y0 = y - y.mean()
    ridge = 1e-6 * max(np.trace(X0.T @ X0) / max(X0.shape[1], 1), 1.0)

```

```

    tangent = np.linalg.solve(X0.T @ X0 + ridge * np.eye(X0.shape[1]), X0.T @
↪y0)
    return tangent / max(np.linalg.norm(tangent), 1e-12)

def orthogonal_unit(tangent, seed):
    rng = np.random.default_rng(int(seed) % (2 ** 32))
    while True:
        vector = rng.normal(size=tangent.shape)
        vector -= np.dot(vector, tangent) * tangent
        norm = np.linalg.norm(vector)
        if norm > 1e-12: return vector / norm

def functional_route_error(actual, predicted, cost):
    return float(entropic_transport_cost(
        torch.tensor(actual, dtype=torch.float32),
        torch.tensor(predicted, dtype=torch.float32),
        cost.float(),
↪
    epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
↪
    iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
    ).mean())

def run_mediation_probe(method, model_seed, fit_frame, eval_frame, cost):
    X_train = stack(fit_frame, "context").astype(float)
    routes_train = stack(fit_frame, "route").astype(float)
    y_train = np.sqrt(np.clip(routes_train, 1e-12, None))
    X_eval = stack(eval_frame, "context").astype(float)
    routes_eval = stack(eval_frame, "route").astype(float)
    y_eval = np.sqrt(np.clip(routes_eval, 1e-12, None))
    tangent = fit_context_tangent(fit_frame)
    orth = orthogonal_unit(tangent, model_seed + 811)

    variants = {
        "full_context": (X_train, X_eval, y_train),
        "angle_shuffled": (X_train, X_eval, y_train[np.random.
↪default_rng(model_seed + 812).permutation(len(y_train))]),
        "tangent_removed": (
            X_train - np.outer(X_train @ tangent, tangent),
            X_eval - np.outer(X_eval @ tangent, tangent),
            y_train,
        ),
        "matched_orthogonal_component": (
            np.outer(X_train @ orth, orth),

```

```

        np.outer(X_eval @ orth, orth),
        y_train,
    ),
}
rows = []
for variant, (train_x, eval_x, train_y) in variants.items():
    probe = Ridge(alpha=float(protocol["mediation_probe"]["ridge_alpha"])).
    ↪fit(train_x, train_y)
    prediction_root = probe.predict(eval_x)
    prediction_route = reconstruct_route_from_root_probe(prediction_root)
    rows.append({
        "model_seed": model_seed, "method": method, "variant": variant,
        "root_route_r2": float(r2_score(y_eval, prediction_root, ↪
    ↪multioutput="variance_weighted")),
        "root_chord_error": float(pairwise_root_chord_np(np.
    ↪vstack([routes_eval, prediction_route]))[:len(routes_eval), len(routes_eval):
    ↪].diagonal().mean()),
        "functional_transport_error": functional_route_error(routes_eval, ↪
    ↪prediction_route, cost),
        "evaluation_samples": len(eval_frame),
    })
return pd.DataFrame(rows)

```

1.11 9. Differential and finite causal mediation

A factor tangent t is fitted on source identities disjoint from causal evaluation. The root-route Jacobian measures local sensitivity:

$$S_t(c) = \|J_q(c)t(c)\|_2, \quad S_\perp(c) = \frac{1}{J} \sum_j \|J_q(c)v_j(c)\|_2,$$

$$\text{DMR} = \frac{\mathbb{E}[S_t]}{\mathbb{E}[S_\perp] + \varepsilon}.$$

Finite interventions compare tangent and orthogonal movements using both nominal and functional route distances. Prediction identity and true-class log probability are audited so a large route displacement cannot be celebrated if it destroys the task.

```

[14]: def orthogonal_directions(tangent, count, seed):
    rng = np.random.default_rng(int(seed) % (2 ** 32)); output = []
    while len(output) < count:
        vector = rng.normal(size=tangent.shape)
        vector -= np.dot(vector, tangent) * tangent
        norm = np.linalg.norm(vector)
        if norm > 1e-12: output.append(vector / norm)
    return output

```

```

def local_tangent(context, direction):
    projected = direction - (context * direction).sum(-1, keepdim=True) *
    ↪ context
    return F.normalize(projected, dim=-1, eps=1e-8)

def root_route_jvp(router, context, direction):
    context = context.detach().requires_grad_(True)
    direction = direction.detach()
    def function(value):
        return torch.sqrt(router(value, temperature=TAU).clamp_min(1e-12))
    _, jvp = torch.autograd.functional.jvp(function, context, direction,
    ↪ create_graph=False)
    return torch.linalg.vector_norm(jvp, dim=-1)

def source_bootstrap_ratio(frame, numerator, denominator, seed):
    rng = np.random.default_rng(int(seed) % (2 ** 32)); n = len(frame)
    values = []
    for _ in range(SOURCE_BOOTSTRAP_SAMPLES):
        sample = frame.iloc[rng.integers(0, n, n)]
        values.append(sample[numerator].mean() / max(sample[denominator].
    ↪ mean(), 1e-12))
    point = frame[numerator].mean() / max(frame[denominator].mean(), 1e-12)
    return point, float(np.quantile(values, 0.025)), float(np.quantile(values,
    ↪ 0.975))

def causal_mediation_probe(model, method, model_seed, fit_frame, cost):
    tangent_np = fit_context_tangent(fit_frame)
    orthogonal_np = orthogonal_directions(
        tangent_np, int(protocol["causal_protocol"]["orthogonal_controls"]),
    ↪ model_seed + 901
    )
    raw_rows = []
    probe_angle = float(protocol["causal_protocol"]["probe_angle"])
    model.eval()
    for x, labels, _, source_ids in fixed_loader(probe_angle, False,
    ↪ causal_eval_indices, False, batch_size=128):
        x, labels = x.to(DEVICE), labels.to(DEVICE)
        with torch.no_grad():
            base = model(x, temperature=TAU)
            base_logp = F.log_softmax(base.logits, -1).gather(1, labels[:,
    ↪ None]).squeeze(1)
            base_pred = base.logits.argmax(1)

```

```

        tangent_global = torch.tensor(tangent_np, dtype=base.context.dtype,
↪device=DEVICE).expand_as(base.context)
        tangent = local_tangent(base.context, tangent_global)
        tangent_sensitivity = root_route_jvp(model.router, base.context,
↪tangent)
        orthogonal_locals = []
        orthogonal_sensitivities = []
        for direction in orthogonal_np:
            global_direction = torch.tensor(direction, dtype=base.context.
↪dtype, device=DEVICE).expand_as(base.context)
            local = local_tangent(base.context, global_direction)
            orthogonal_locals.append(local)
            orthogonal_sensitivities.append(root_route_jvp(model.router, base.
↪context, local))
        orthogonal_sensitivity = torch.stack(orthogonal_sensitivities).mean(0)

        for scale in protocol["causal_protocol"]["intervention_scales"]:
            scale = float(scale)
            with torch.no_grad():
                tangent_context = F.normalize(base.context + scale * tangent,
↪dim=-1)
                tangent_route = model.route_context(tangent_context,
↪temperature=TAU)
                nominal_t = root_simplex_chord(tangent_route, base.route)
                functional_t = entropic_transport_cost(
                    tangent_route, base.route, cost.to(DEVICE),
                    ↪
↪epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
                    ↪
↪iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
                )
                z = torch.einsum("bh,bhd->bd", tangent_route, base.host_outputs)
                new_logits = model.classifier(model.synthesis_norm(z))
                new_logp = F.log_softmax(new_logits, -1).gather(1, labels[:,
↪None]).squeeze(1)
                new_pred = new_logits.argmax(1)

                nominal_controls = []; functional_controls = []
                for local in orthogonal_locals:
                    control_context = F.normalize(base.context + scale * local,
↪dim=-1)
                    control_route = model.route_context(control_context,
↪temperature=TAU)
                    nominal_controls.append(root_simplex_chord(control_route,
↪base.route))
                    functional_controls.append(entropic_transport_cost(

```

```

        control_route, base.route, cost.to(DEVICE),
        □
↪epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
        □
↪iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
        ))
        nominal_o = torch.stack(nominal_controls).mean(0)
        functional_o = torch.stack(functional_controls).mean(0)

    for index, source_id in enumerate(source_ids):
        raw_rows.append({
            "model_seed": model_seed, "method": method, "source_index":□
↪int(source_id),
            "scale": scale,
            "nominal_tangent_change": float(nominal_t[index].cpu()),
            "nominal_orthogonal_change": float(nominal_o[index].cpu()),
            "functional_tangent_change": float(functional_t[index].
↪cpu()),
            "functional_orthogonal_change": float(functional_o[index].
↪cpu()),
            "tangent_jacobian_sensitivity":□
↪float(tangent_sensitivity[index].detach().cpu()),
            "orthogonal_jacobian_sensitivity":□
↪float(orthogonal_sensitivity[index].detach().cpu()),
            "prediction_preserved": int(base_pred[index] ==□
↪new_pred[index]),
            "true_class_log_prob_change": float((new_logp[index] -□
↪base_logp[index]).cpu()),
        })
    raw = pd.DataFrame(raw_rows)
    summary = []
    for scale, block in raw.groupby("scale"):
        nominal = source_bootstrap_ratio(block, "nominal_tangent_change",□
↪"nominal_orthogonal_change", model_seed + int(abs(scale) * 1000) + 17)
        functional = source_bootstrap_ratio(block, "functional_tangent_change",□
↪"functional_orthogonal_change", model_seed + int(abs(scale) * 1000) + 27)
        dmr = source_bootstrap_ratio(block, "tangent_jacobian_sensitivity",□
↪"orthogonal_jacobian_sensitivity", model_seed + 37)
        preservation = bootstrap_source_mean(block.prediction_preserved.
↪to_numpy(float), model_seed + 47)
        summary.append({
            "model_seed": model_seed, "method": method, "scale": scale,
            "nominal_csr": nominal[0], "nominal_csr_ci_low": nominal[1],□
↪"nominal_csr_ci_high": nominal[2],
            "functional_csr": functional[0], "functional_csr_ci_low":□
↪functional[1], "functional_csr_ci_high": functional[2],

```



```

        "dmr": dmr[0], "dmr_ci_low": dmr[1], "dmr_ci_high": dmr[2],
        "prediction_preservation": preservation[0],
        "prediction_preservation_ci_low": preservation[1],
        "prediction_preservation_ci_high": preservation[2],
        "mean_true_class_log_prob_change": block.true_class_log_prob_change.
    ↪mean(),
    })
    return raw, pd.DataFrame(summary)

```

1.12 10. Host ecology and ablation

Route-mass differentiation is not enough. A host is functionally specialized only when removing it produces a factor-dependent effect. The notebook therefore exports territory curves, entropy specialization, overlap, functional cost, all-host ablations, top-host removal, and a deterministic random-host control.

```

[15]: @torch.no_grad()
def host_ecology_tables(model, method, model_seed, dense_trace, cost):
    routes = torch.tensor(stack(dense_trace, "route"), dtype=torch.float32)
    factors = torch.tensor(dense_trace.angle.to_numpy(float), dtype=torch.
    ↪float32)
    unique, territory = host_territory(routes, factors)
    specialization = host_specialization(territory)
    overlap = territory_overlap(territory)
    territory_rows = []
    for factor_index, angle in enumerate(unique.tolist()):
        for host in range(NUM_HOSTS):
            territory_rows.append({
                "model_seed": model_seed, "method": method, "angle": angle,
                "host": host, "mean_route_mass": float(territory[factor_index,
    ↪host]),
                "host_specialization": float(specialization[host]),
            })
    overlap_rows = [
        {"model_seed": model_seed, "method": method, "host_a": a, "host_b": b,
        "territory_overlap": float(overlap[a, b]), "functional_cost":
    ↪float(cost[a, b])}
        for a in range(NUM_HOSTS) for b in range(NUM_HOSTS)
    ]

    ablation_rows = []
    rng = np.random.default_rng(model_seed + 1110)
    for angle in DENSE_EVAL_ANGLES:
        angle_territory = territory[torch.argmax(torch.abs(unique - angle))]
        top_host = int(torch.argmax(angle_territory))
        random_host = int(rng.integers(0, NUM_HOSTS))
        total = 0; baseline_correct = 0

```

```

per_host_correct = np.zeros(NUM_HOSTS, dtype=int)
for x, y, _, _ in fixed_loader(angle, False, causal_eval_indices,
↪False, batch_size=128):
    x, y = x.to(DEVICE), y.to(DEVICE)
    trace = model(x, temperature=TAU)
    baseline_correct += int((trace.logits.argmax(1) == y).sum()); total
↪+= y.numel()
    for host in range(NUM_HOSTS):
        route = trace.route.clone(); route[:, host] = 0.0
        route = route / route.sum(-1, keepdim=True).clamp_min(1e-12)
        z = torch.einsum("bh,bhd->bd", route, trace.host_outputs)
        logits = model.classifier(model.synthesis_norm(z))
        per_host_correct[host] += int((logits.argmax(1) == y).sum())
baseline_accuracy = baseline_correct / total
for host in range(NUM_HOSTS):
    ablated_accuracy = per_host_correct[host] / total
    ablation_rows.append({
        "model_seed": model_seed, "method": method, "angle": angle,
↪"host": host,
        "baseline_accuracy": baseline_accuracy, "ablated_accuracy":
↪ablated_accuracy,
        "accuracy_drop": baseline_accuracy - ablated_accuracy,
        "is_top_host": int(host == top_host), "is_random_control":
↪int(host == random_host),
    })
return pd.DataFrame(territory_rows), pd.DataFrame(overlap_rows), pd.
↪DataFrame(ablation_rows)

```

1.13 11. Per-seed execution

Each seed performs two explicit phases:

1. reproduce and freeze one aligned context checkpoint;
2. train R0, R1, and R2 from matched host/classifier states.

The seed then builds method-specific host cost matrices and runs geometry, probes, causal interventions, and ablations.

```

[16]: def source_geometry_summary(frame, method, model_seed, cost):
    rows = []
    for partition, angles in {"trained": TRAIN_ANGLES, "heldout_dense":
↪HELDOUT_ANGLES}.items():
        sub = frame[frame.angle.isin(angles)]
        nominal = grouped_route_geometry(sub, cost=None)
        functional = grouped_route_geometry(sub, cost=cost)
        for name, table in [("nominal", nominal), ("functional", functional)]:
            for metric in ["rho", "stress"]:

```

```

        mean, low, high = bootstrap_source_mean(table[metric],
↪model_seed + (11 if metric == "rho" else 12))
        rows.append({
            "model_seed": model_seed, "method": method, "partition":
↪partition,
            "geometry": name, "metric": metric, "mean": mean,
            "source_ci_low": low, "source_ci_high": high,
            "source_blocks": len(table),
        })
    return pd.DataFrame(rows)

def paired_source_geometry_effect(source_tables, treatment, control,
↪model_seed):
    rows = []
    for partition in ["trained", "heldout_dense"]:
        for geometry in ["nominal", "functional"]:
            for metric in ["rho", "stress"]:
                t = source_tables[(treatment, partition,
↪geometry)][["source_index", metric]]
                c = source_tables[(control, partition,
↪geometry)][["source_index", metric]]
                merged = t.merge(c, on="source_index", suffixes=("_t", "_c"))
                delta = merged[f"{metric}_t"] - merged[f"{metric}_c"]
                mean, low, high = bootstrap_source_mean(delta, model_seed + 211)
                rows.append({
                    "model_seed": model_seed, "contrast":
↪f"{treatment}_vs_{control}",
                    "partition": partition, "geometry": geometry, "metric":
↪metric,
                    "delta_mean": mean, "delta_ci_low": low, "delta_ci_high":
↪high,
                    "source_blocks": len(merged),
                })
    return rows

def run_seed(model_seed):
    print(f"\n===== MODEL SEED {model_seed} =====")
    random.seed(model_seed); np.random.seed(model_seed); torch.
↪manual_seed(model_seed)
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(model_seed)

    source_model, checkpoint, checkpoint_hash, context_stage =
↪train_aligned_context_checkpoint(model_seed)

```

```

    context_bank = collect_context_bank(source_model, TRAIN_ANGLES,
↪train_indices)
    common_state = create_common_host_state(model_seed)
    common_state_hash = state_hash({
        **{f"hosts.{k}": v for k, v in common_state["hosts"].items()},
        **{f"synthesis_norm.{k}": v for k, v in common_state["synthesis_norm"].
↪items()},
        **{f"classifier.{k}": v for k, v in common_state["classifier"].items()},
    })

    models = {}; stage_tables = []; evaluation_tables = []; router_init_rows =
↪[]
    for method in METHODS:
        print(f"\n--- {method} ---")
        model, init_diag = build_routing_model(method, checkpoint,
↪common_state, context_bank, model_seed)
        router_init_rows.append({
            "model_seed": model_seed, "method": method,
            "router_parameters": parameter_count(model.router), **init_diag,
        })
        model, stage, evaluation = train_router_treatment(model_seed, method,
↪model, checkpoint_hash)
        models[method] = model; stage_tables.append(stage); evaluation_tables.
↪append(evaluation)

    stage = pd.concat(stage_tables, ignore_index=True)
    evaluation = pd.concat(evaluation_tables, ignore_index=True)
    for stage_id in sorted(stage.stage.unique()):
        block = stage[stage.stage == stage_id]
        assert block.forward_images.nunique() == 1 and block.optimizer_steps.
↪nunique() == 1

    dense_trace = pd.concat([
        collect_trace(models[method], method, model_seed, DENSE_EVAL_ANGLES,
↪test_indices)
        for method in METHODS
    ], ignore_index=True)
    probe_fit_trace = pd.concat([
        collect_trace(models[method], method, model_seed, TRAIN_ANGLES,
↪probe_fit_indices)
        for method in METHODS
    ], ignore_index=True)
    probe_eval_trace = pd.concat([
        collect_trace(models[method], method, model_seed, HELDOUT_ANGLES,
↪probe_eval_indices)
        for method in METHODS

```

```

    ], ignore_index=True)
    tangent_fit_trace = pd.concat([
        collect_trace(models[method], method, model_seed, TRAIN_ANGLES,
↪tangent_fit_indices)
        for method in METHODS
    ], ignore_index=True)
    context_preservation = context_preservation_table(dense_trace)

    cost_rows = []; geometry_rows = []; probe_rows = []; causal_raw = [];
↪causal_summary = []
    territory_rows = []; overlap_rows = []; ablation_rows = [];
↪source_geometry_tables = {}
    method_summary_rows = []; forgetting_rows = []
    final_stage = int(evaluation.stage.max())

    for method in METHODS:
        model = models[method]
        method_dense = dense_trace[dense_trace.method == method]
        cost = build_host_cost(model, DENSE_EVAL_ANGLES, metric_indices)
        for a in range(NUM_HOSTS):
            for b in range(NUM_HOSTS):
                cost_rows.append({"model_seed": model_seed, "method": method,
↪"host_a": a, "host_b": b, "cost": float(cost[a, b])})

        # Preserve source-level tables for paired contrasts.
        for partition, angles in {"trained": TRAIN_ANGLES, "heldout_dense":
↪HELDOUT_ANGLES}.items():
            sub = method_dense[method_dense.angle.isin(angles)]
            source_geometry_tables[(method, partition, "nominal")] =
↪grouped_route_geometry(sub, cost=None)
            source_geometry_tables[(method, partition, "functional")] =
↪grouped_route_geometry(sub, cost=cost)
            geometry_rows.append(source_geometry_summary(method_dense, method,
↪model_seed, cost))

        fit = probe_fit_trace[probe_fit_trace.method == method]
        eval_frame = probe_eval_trace[probe_eval_trace.method == method]
        probe_rows.append(run_mediation_probe(method, model_seed, fit,
↪eval_frame, cost))
        causal_fit = tangent_fit_trace[tangent_fit_trace.method == method]
        raw, summary = causal_mediation_probe(model, method, model_seed,
↪causal_fit, cost)
        causal_raw.append(raw); causal_summary.append(summary)
        territory, overlap, ablation = host_ecology_tables(model, method,
↪model_seed, method_dense, cost)

```

```

territory_rows.append(territory); overlap_rows.append(overlap);
↪ablation_rows.append(ablation)

method_eval = evaluation[evaluation.method == method]
for angle in TRAIN_ANGLES:
    seen_stage = CURRICULUM.index(angle)
    rows = method_eval[(method_eval.angle == angle) & (method_eval.
↪stage >= seen_stage)]
    best = float(rows.accuracy.max()); final_acc = float(rows[rows.
↪stage == final_stage].accuracy.iloc[0])
    forgetting_rows.append({
        "model_seed": model_seed, "method": method, "angle": angle,
        "best_accuracy": best, "final_accuracy": final_acc,
↪"forgetting": best - final_acc,
    })
    final = method_eval[method_eval.stage == final_stage]
    nominal_train = source_geometry_tables[(method, "trained", "nominal")]
    functional_train = source_geometry_tables[(method, "trained",
↪"functional")]
    method_summary_rows.append({
        "model_seed": model_seed, "method": method,
        "nominal_route_rho": nominal_train.rho.mean(),
        "functional_route_rho": functional_train.rho.mean(),
        "mean_heldout_accuracy": final[final.angle.isin(HELDOUT_ANGLES)].
↪accuracy.mean(),
        "final_trained_accuracy": final[final.angle.isin(TRAIN_ANGLES)].
↪accuracy.mean(),
        "router_parameters": parameter_count(model.router),
    })

    paired_rows = []
    for contrast in PRIMARY_CONTRASTS:
        treatment = contrast["treatment"]; control = contrast["control"]
        paired_rows.
↪extend(paired_source_geometry_effect(source_geometry_tables, treatment,
↪control, model_seed))

    forgetting = pd.DataFrame(forgetting_rows)
    method_summary = pd.DataFrame(method_summary_rows).merge(
        forgetting.groupby(["model_seed", "method"], as_index=False).forgetting.
↪mean().rename(columns={"forgetting": "mean_forgetting"}),
        on=["model_seed", "method"], how="left",
    )

    return {
        "checkpoint_hash": checkpoint_hash,

```

```

        "common_state_hash": common_state_hash,
        "context_stage": context_stage,
        "router_initialization": pd.DataFrame(router_init_rows),
        "stage": stage,
        "evaluation": evaluation,
        "trace": dense_trace,
        "context_preservation": context_preservation,
        "host_cost": pd.DataFrame(cost_rows),
        "geometry": pd.concat(geometry_rows, ignore_index=True),
        "paired_geometry": pd.DataFrame(paired_rows),
        "mediation_probe": pd.concat(probe_rows, ignore_index=True),
        "causal_raw": pd.concat(causal_raw, ignore_index=True),
        "causal_summary": pd.concat(causal_summary, ignore_index=True),
        "territory": pd.concat(territory_rows, ignore_index=True),
        "overlap": pd.concat(overlap_rows, ignore_index=True),
        "ablation": pd.concat(ablation_rows, ignore_index=True),
        "forgetting": forgetting,
        "method_summary": method_summary,
    }

all_runs = []
for model_seed in MODEL_SEEDS:
    all_runs.append((model_seed, run_seed(model_seed)))
print("Completed seeds:", [seed for seed, _ in all_runs])

```

===== MODEL SEED 0 =====

```

context {'model_seed': 0, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 2.632627371000126, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.8274404928088188, 'current_ce':
1.7848660051822662, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,
'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8489894159138203}
context {'model_seed': 0, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 4.401491800000258, 'forward_images': 2048,
'optimizer_steps': 16, 'loss': 1.556731328368187, 'current_ce':
1.3552666492760181, 'anchor_ce': 1.2216982170939445, 'route_geo':
0.12866627983748913, 'context_geo': 0.04250857466831803, 'context_spread':
0.00020963217411917867, 'fiber': 0.5014275535941124, 'centroid':
0.01165978214999086, 'host_diversity': 0.7302504368126392}
context {'model_seed': 0, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 3.2705473440000787, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.7175729237496853, 'current_ce':
0.5171247823163867, 'anchor_ce': 1.152430597692728, 'route_geo':
0.2539558978751302, 'context_geo': 0.050096603110432625, 'context_spread':
3.274957009580248e-05, 'fiber': 0.47606404684484005, 'centroid':
0.036922108905855566, 'host_diversity': 0.5829776097089052}

```

```

context {'model_seed': 0, 'stage': 3, 'current_angle': 60.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 2.891647551999995,
'forward_images': 4096, 'optimizer_steps': 16, 'loss': 2.0344742611050606,
'current_ce': 1.8286719769239426, 'anchor_ce': 1.183510396629572, 'route_geo':
0.3343885689973831, 'context_geo': 0.07600971916690469, 'context_spread':
5.0942230700457e-07, 'fiber': 0.429350771009922, 'centroid':
0.09437775751575828, 'host_diversity': 0.4044990334659815}
context {'model_seed': 0, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 4.155134855000142,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.8970313258469105,
'current_ce': 0.6945051997900009, 'anchor_ce': 1.2496512308716774, 'route_geo':
0.2352123148739338, 'context_geo': 0.07707270327955484, 'context_spread': 0.0,
'fiber': 0.4996451511979103, 'centroid': 0.07309799338690937, 'host_diversity':
0.3539069704711437}

```

--- r0_mlp ---

```

{'model_seed': 0, 'method': 'r0_mlp', 'stage': 0, 'current_angle': -60.0,
'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',
'router_parameters': 4740, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 1.3189341869992859, 'loss': 1.9552126452326775, 'current_ce':
1.9127385467290878, 'anchor_ce': 0.0, 'balance': 0.0015202918330032844,
'host_diversity': 0.8479611836373806}
{'model_seed': 0, 'method': 'r0_mlp', 'stage': 1, 'current_angle': -30.0,
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',
'router_parameters': 4740, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 3.4304530610006623, 'loss': 1.5259546600282192, 'current_ce':
1.3626900389790535, 'anchor_ce': 1.2748901322484016, 'balance':
0.0058094225387321785, 'host_diversity': 0.7097023166716099}
{'model_seed': 0, 'method': 'r0_mlp', 'stage': 2, 'current_angle': 0.0,
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',
'router_parameters': 4740, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.1417082860007213, 'loss': 0.7164063062518835, 'current_ce':
0.5718985665589571, 'anchor_ce': 1.148888349533081, 'balance':
0.014607940218411386, 'host_diversity': 0.5777700282633305}
{'model_seed': 0, 'method': 'r0_mlp', 'stage': 3, 'current_angle': 60.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',
'router_parameters': 4740, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 3.868907297000078, 'loss': 2.031250573694706, 'current_ce':
1.894211307168007, 'anchor_ce': 1.1517366655170918, 'balance':
0.03449837467633188, 'host_diversity': 0.4028140436857939}
{'model_seed': 0, 'method': 'r0_mlp', 'stage': 4, 'current_angle': 30.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',
'router_parameters': 4740, 'forward_images': 5120, 'optimizer_steps': 16,

```



```
'wall_seconds': 4.598745182999664, 'loss': 0.8533009178936481, 'current_ce':  
0.706902788951993, 'anchor_ce': 1.2490266114473343, 'balance':  
0.04499077261425555, 'host_diversity': 0.3849184177815914}
```

--- r1_linear ---

```
{'model_seed': 0, 'method': 'r1_linear', 'stage': 0, 'current_angle': -60.0,  
'seen_angles': '[-60.0]', 'context_checkpoint_hash':  
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',  
'router_parameters': 20, 'forward_images': 1024, 'optimizer_steps': 16,  
'wall_seconds': 0.9900351400001455, 'loss': 1.953853115439415, 'current_ce':  
1.9095775559544563, 'anchor_ce': 0.0, 'balance': 0.04739145073108375,  
'host_diversity': 0.8381193093955517}  
{'model_seed': 0, 'method': 'r1_linear', 'stage': 1, 'current_angle': -30.0,  
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':  
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',  
'router_parameters': 20, 'forward_images': 2048, 'optimizer_steps': 16,  
'wall_seconds': 3.838348735999716, 'loss': 1.5187577046453953, 'current_ce':  
1.3573283404111862, 'anchor_ce': 1.2624009028077126, 'balance':  
0.03891276312060654, 'host_diversity': 0.6648724786937237}  
{'model_seed': 0, 'method': 'r1_linear', 'stage': 2, 'current_angle': 0.0,  
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':  
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',  
'router_parameters': 20, 'forward_images': 3072, 'optimizer_steps': 16,  
'wall_seconds': 2.1171459030001643, 'loss': 0.7034838777035475, 'current_ce':  
0.5643058251589537, 'anchor_ce': 1.1302739940583706, 'balance':  
0.030536100501194596, 'host_diversity': 0.49247700348496437}  
{'model_seed': 0, 'method': 'r1_linear', 'stage': 3, 'current_angle': 60.0,  
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':  
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',  
'router_parameters': 20, 'forward_images': 4096, 'optimizer_steps': 16,  
'wall_seconds': 3.653433481999855, 'loss': 2.0404341369867325, 'current_ce':  
1.9119092151522636, 'anchor_ce': 1.1273665092885494, 'balance':  
0.027588088647462428, 'host_diversity': 0.28817639499902725}  
{'model_seed': 0, 'method': 'r1_linear', 'stage': 4, 'current_angle': 30.0,  
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':  
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',  
'router_parameters': 20, 'forward_images': 5120, 'optimizer_steps': 16,  
'wall_seconds': 6.771339691000321, 'loss': 0.8382679857313633, 'current_ce':  
0.6992356162518263, 'anchor_ce': 1.2307197749614716, 'balance':  
0.024255713215097785, 'host_diversity': 0.29495195858180523}
```

--- r2_prototype_energy ---

```
{'model_seed': 0, 'method': 'r2_prototype_energy', 'stage': 0, 'current_angle':  
-60.0, 'seen_angles': '[-60.0]', 'context_checkpoint_hash':  
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',  
'router_parameters': 24, 'forward_images': 1024, 'optimizer_steps': 16,  
'wall_seconds': 1.4703583119999166, 'loss': 1.9376520365476608, 'current_ce':  
1.8886029049754143, 'anchor_ce': 0.0, 'balance': 0.19925779197365046,
```

```

'host_diversity': 0.7817241512238979}
{'model_seed': 0, 'method': 'r2_prototype_energy', 'stage': 1, 'current_angle':
-30.0, 'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',
'router_parameters': 24, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.5583437409995895, 'loss': 1.5058800131082535, 'current_ce':
1.3543913587927818, 'anchor_ce': 1.2256297916173935, 'balance':
0.06120363459922373, 'host_diversity': 0.5173102505505085}
{'model_seed': 0, 'method': 'r2_prototype_energy', 'stage': 2, 'current_angle':
0.0, 'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',
'router_parameters': 24, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 3.092049658999713, 'loss': 0.6291177216917276, 'current_ce':
0.5016073156148195, 'anchor_ce': 1.0952593237161636, 'balance':
0.01959826349047944, 'host_diversity': 0.34009116142988205}
{'model_seed': 0, 'method': 'r2_prototype_energy', 'stage': 3, 'current_angle':
60.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',
'router_parameters': 24, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 2.7051864039995053, 'loss': 2.1778765618801117, 'current_ce':
2.0698625445365906, 'anchor_ce': 0.9510273709893227, 'balance':
0.005153058125870302, 'host_diversity': 0.25307262130081654}
{'model_seed': 0, 'method': 'r2_prototype_energy', 'stage': 4, 'current_angle':
30.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]',
'context_checkpoint_hash':
'994fbd92290ab0dba14d5f0f3dd615a8b77dbbf53659e986fd4cc050984cebcd',
'router_parameters': 24, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 5.0006537609997395, 'loss': 0.8730112574994564, 'current_ce':
0.7464463226497173, 'anchor_ce': 1.1486669927835464, 'balance':
0.0024948157861217624, 'host_diversity': 0.2314701322466135}

```

===== MODEL SEED 1 =====

```

context {'model_seed': 1, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 1.0197846979999667, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.9558866396546364, 'current_ce':
1.9131375774741173, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,
'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8524815179407597}
context {'model_seed': 1, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 1.6695182200001, 'forward_images': 2048,
'optimizer_steps': 16, 'loss': 1.6175028681755066, 'current_ce':
1.412396915256977, 'anchor_ce': 1.2397688403725624, 'route_geo':
0.12748053157702088, 'context_geo': 0.039695704937912524, 'context_spread':
0.0002491801969881635, 'fiber': 0.5564186070114374, 'centroid':
0.013868008594727144, 'host_diversity': 0.7176944762468338}
context {'model_seed': 1, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 2.814521892999437, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.7259656488895416, 'current_ce':

```

```

0.5248025851324201, 'anchor_ce': 1.1940825879573822, 'route_geo':
0.23109073750674725, 'context_geo': 0.047820870066061616, 'context_spread':
3.415597138256565e-05, 'fiber': 0.47823939844965935, 'centroid':
0.02468682902690489, 'host_diversity': 0.574312187731266}
context {'model_seed': 1, 'stage': 3, 'current_angle': 60.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 5.599867234000158,
'forward_images': 4096, 'optimizer_steps': 16, 'loss': 2.1036443263292313,
'current_ce': 1.887081801891327, 'anchor_ce': 1.2245997302234173, 'route_geo':
0.3739459365606308, 'context_geo': 0.09927346231415868, 'context_spread':
2.444048291749823e-06, 'fiber': 0.4764918442815542, 'centroid':
0.06965764000779018, 'host_diversity': 0.38946177437901497}
context {'model_seed': 1, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 3.8619555260002016,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.9296926483511925,
'current_ce': 0.7216502167284489, 'anchor_ce': 1.2836312800645828, 'route_geo':
0.2755517475306988, 'context_geo': 0.07046858500689268, 'context_spread': 0.0,
'fiber': 0.532947288826108, 'centroid': 0.019987852196209133, 'host_diversity':
0.34861025772988796}

```

--- r0_mlp ---

```

{'model_seed': 1, 'method': 'r0_mlp', 'stage': 0, 'current_angle': -60.0,
'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 4740, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 0.9820993370003634, 'loss': 1.9639067947864532, 'current_ce':
1.9206319749355316, 'anchor_ce': 0.0, 'balance': 0.0027901982466573827,
'host_diversity': 0.8627063930034637}
{'model_seed': 1, 'method': 'r0_mlp', 'stage': 1, 'current_angle': -30.0,
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 4740, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.842850520999491, 'loss': 1.6193230673670769, 'current_ce':
1.4581156484782696, 'anchor_ce': 1.2333434112370014, 'balance':
0.005056660971604288, 'host_diversity': 0.7524046488106251}
{'model_seed': 1, 'method': 'r0_mlp', 'stage': 2, 'current_angle': 0.0,
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 4740, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.1149404190000496, 'loss': 0.6948736533522606, 'current_ce':
0.5469145150855184, 'anchor_ce': 1.1804895624518394, 'balance':
0.0023679269361309707, 'host_diversity': 0.5958361085504293}
{'model_seed': 1, 'method': 'r0_mlp', 'stage': 3, 'current_angle': 60.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 4740, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 3.3392903929998283, 'loss': 2.0649685487151146, 'current_ce':
1.9216469451785088, 'anchor_ce': 1.2269722372293472, 'balance':
0.0014205657898855861, 'host_diversity': 0.4110671915113926}

```

```

{'model_seed': 1, 'method': 'r0_mlp', 'stage': 4, 'current_angle': 30.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 4740, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 3.503174900999511, 'loss': 0.8886798247694969, 'current_ce':
0.7383868508040905, 'anchor_ce': 1.2898689657449722, 'balance':
0.0023046408605296165, 'host_diversity': 0.42381662875413895}

--- r1_linear ---
{'model_seed': 1, 'method': 'r1_linear', 'stage': 0, 'current_angle': -60.0,
'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 20, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 0.9667877250003585, 'loss': 1.9635368436574936, 'current_ce':
1.9198649302124977, 'anchor_ce': 0.0, 'balance': 0.013371559558436275,
'host_diversity': 0.8600672297179699}
{'model_seed': 1, 'method': 'r1_linear', 'stage': 1, 'current_angle': -30.0,
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 20, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.5483037020003394, 'loss': 1.61605204641819, 'current_ce':
1.4556906558573246, 'anchor_ce': 1.2291380874812603, 'balance':
0.01200243312632665, 'host_diversity': 0.7369490414857864}
{'model_seed': 1, 'method': 'r1_linear', 'stage': 2, 'current_angle': 0.0,
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 20, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.824627373000112, 'loss': 0.6972067542374134, 'current_ce':
0.5504010552540421, 'anchor_ce': 1.1747403219342232, 'balance':
0.008589686243794858, 'host_diversity': 0.5780437644571066}
{'model_seed': 1, 'method': 'r1_linear', 'stage': 3, 'current_angle': 60.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 20, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 2.7001684679999016, 'loss': 2.055703803896904, 'current_ce':
1.9155749157071114, 'anchor_ce': 1.2066252902150154, 'balance':
0.0062353325192816556, 'host_diversity': 0.3830923344939947}
{'model_seed': 1, 'method': 'r1_linear', 'stage': 4, 'current_angle': 30.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 20, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 3.8584734339992792, 'loss': 0.8602291084825993, 'current_ce':
0.7146435491740704, 'anchor_ce': 1.2733596116304398, 'balance':
0.006384415930369869, 'host_diversity': 0.35860748030245304}

--- r2_prototype_energy ---
{'model_seed': 1, 'method': 'r2_prototype_energy', 'stage': 0, 'current_angle':
-60.0, 'seen_angles': '[-60.0]', 'context_checkpoint_hash':

```

```

'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 24, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 0.9858306179994543, 'loss': 1.891967162489891, 'current_ce':
1.8482766151428223, 'anchor_ce': 0.0, 'balance': 0.09636690095067024,
'host_diversity': 0.7774441614747047}
{'model_seed': 1, 'method': 'r2_prototype_energy', 'stage': 1, 'current_angle':
-30.0, 'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 24, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 2.0973870999996507, 'loss': 1.5087724961340427, 'current_ce':
1.3591883182525635, 'anchor_ce': 1.2199189588427544, 'balance':
0.04956768825650215, 'host_diversity': 0.5022782851010561}
{'model_seed': 1, 'method': 'r2_prototype_energy', 'stage': 2, 'current_angle':
0.0, 'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 24, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.1251182440000775, 'loss': 0.604645324870944, 'current_ce':
0.4641111632809043, 'anchor_ce': 1.2203659638762474, 'balance':
0.044029265525750816, 'host_diversity': 0.3259218968451023}
{'model_seed': 1, 'method': 'r2_prototype_energy', 'stage': 3, 'current_angle':
60.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 24, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 3.7240538779997223, 'loss': 2.0638401582837105, 'current_ce':
1.9441641494631767, 'anchor_ce': 1.0686484277248383, 'balance':
0.016885787539649755, 'host_diversity': 0.2393366601318121}
{'model_seed': 1, 'method': 'r2_prototype_energy', 'stage': 4, 'current_angle':
30.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]',
'context_checkpoint_hash':
'69491e17d8bbca1e8038b4ba470ff9d794f19a9c4bfbade32cc0422e38b17a7f',
'router_parameters': 24, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 3.3454793239998253, 'loss': 0.9339932538568974, 'current_ce':
0.794768538326025, 'anchor_ce': 1.2774275541305542, 'balance':
0.014789444219786674, 'host_diversity': 0.2148495763540268}

```

===== MODEL SEED 2 =====

```

context {'model_seed': 2, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 1.0795255760003784, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.947095662355423, 'current_ce':
1.9044951051473618, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,
'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8495110683143139}
context {'model_seed': 2, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 1.609672600000522, 'forward_images': 2048,
'optimizer_steps': 16, 'loss': 1.5397205278277397, 'current_ce':
1.3412109091877937, 'anchor_ce': 1.2136708721518517, 'route_geo':
0.1188340550288558, 'context_geo': 0.029194870847277343, 'context_spread':
0.00015080858565852395, 'fiber': 0.5210066586732864, 'centroid':

```

```

0.012672376593400259, 'host_diversity': 0.7129624262452126}
context {'model_seed': 2, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 2.205050690000462, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.700000949203968, 'current_ce':
0.5038357488811016, 'anchor_ce': 1.1804726421833038, 'route_geo':
0.20282090362161398, 'context_geo': 0.04489377071149647, 'context_spread':
1.4217901815882783e-05, 'fiber': 0.4760640971362591, 'centroid':
0.019051062859944068, 'host_diversity': 0.5717995557934046}
context {'model_seed': 2, 'stage': 3, 'current_angle': 60.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 7.783889790000103,
'forward_images': 4096, 'optimizer_steps': 16, 'loss': 2.05204014480114,
'current_ce': 1.8358505442738533, 'anchor_ce': 1.1888744793832302, 'route_geo':
0.36272923089563847, 'context_geo': 0.1138483565300703, 'context_spread':
7.844577498104799e-07, 'fiber': 0.4719220530241728, 'centroid':
0.1457324679940939, 'host_diversity': 0.3752317223697901}
context {'model_seed': 2, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 6.9928837650004425,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.902299165725708,
'current_ce': 0.6928039211779833, 'anchor_ce': 1.2419192790985107, 'route_geo':
0.2993121109902859, 'context_geo': 0.07140571461059153, 'context_spread': 0.0,
'fiber': 0.5321044810116291, 'centroid': 0.06148658273741603, 'host_diversity':
0.3710397854447365}

```

--- r0_mlp ---

```

{'model_seed': 2, 'method': 'r0_mlp', 'stage': 0, 'current_angle': -60.0,
'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 4740, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 1.4708330489993386, 'loss': 1.8967962339520454, 'current_ce':
1.854620672762394, 'anchor_ce': 0.0, 'balance': 0.00342112811813422,
'host_diversity': 0.8400899544358253}
{'model_seed': 2, 'method': 'r0_mlp', 'stage': 1, 'current_angle': -30.0,
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 4740, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 2.746851545000027, 'loss': 1.5609856098890305, 'current_ce':
1.404901523143053, 'anchor_ce': 1.1976505890488625, 'balance':
0.00747344849514775, 'host_diversity': 0.718907181173563}
{'model_seed': 2, 'method': 'r0_mlp', 'stage': 2, 'current_angle': 0.0,
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 4740, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.099265027999536, 'loss': 0.6792748104780912, 'current_ce':
0.5325889708474278, 'anchor_ce': 1.1652431599795818, 'balance':
0.010298439825419337, 'host_diversity': 0.5929319337010384}
{'model_seed': 2, 'method': 'r0_mlp', 'stage': 3, 'current_angle': 60.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',

```

```

'router_parameters': 4740, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 4.004942288000166, 'loss': 2.0040471106767654, 'current_ce':
1.8637629747390747, 'anchor_ce': 1.1853099502623081, 'balance':
0.016685493290424347, 'host_diversity': 0.4183771647512913}
{'model_seed': 2, 'method': 'r0_mlp', 'stage': 4, 'current_angle': 30.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 4740, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 3.9902339840000423, 'loss': 0.860228955745697, 'current_ce':
0.7094339206814766, 'anchor_ce': 1.3075628131628036, 'balance':
0.039516587974503636, 'host_diversity': 0.3612586446106434}

```

--- r1_linear ---

```

{'model_seed': 2, 'method': 'r1_linear', 'stage': 0, 'current_angle': -60.0,
'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 20, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 1.4500259090000327, 'loss': 1.8926045671105385, 'current_ce':
1.8491422683000565, 'anchor_ce': 0.0, 'balance': 0.0348273377167061,
'host_diversity': 0.8344187773764133}
{'model_seed': 2, 'method': 'r1_linear', 'stage': 1, 'current_angle': -30.0,
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 20, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.54217206300018, 'loss': 1.556343100965023, 'current_ce':
1.4009239934384823, 'anchor_ce': 1.1934746652841568, 'balance':
0.021321702399291098, 'host_diversity': 0.7001113258302212}
{'model_seed': 2, 'method': 'r1_linear', 'stage': 2, 'current_angle': 0.0,
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 20, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.7676391349996265, 'loss': 0.6687247306108475, 'current_ce':
0.5248356368392706, 'anchor_ce': 1.1569824032485485, 'balance':
0.012445943721104413, 'host_diversity': 0.5513710621744394}
{'model_seed': 2, 'method': 'r1_linear', 'stage': 3, 'current_angle': 60.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 20, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 3.2263046939997366, 'loss': 2.0318883061408997, 'current_ce':
1.8979027569293976, 'anchor_ce': 1.15117646753788, 'balance':
0.012596969667356461, 'host_diversity': 0.3647615686058998}
{'model_seed': 2, 'method': 'r1_linear', 'stage': 4, 'current_angle': 30.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 20, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 3.9270569189993694, 'loss': 0.8344392813742161, 'current_ce':
0.6934467032551765, 'anchor_ce': 1.2471576407551765, 'balance':
0.01148446643492207, 'host_diversity': 0.314051853492856}

```

```

--- r2_prototype_energy ---
{'model_seed': 2, 'method': 'r2_prototype_energy', 'stage': 0, 'current_angle':
-60.0, 'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 24, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 1.4488059520008392, 'loss': 1.8413704186677933, 'current_ce':
1.7984515130519867, 'anchor_ce': 0.0, 'balance': 0.09407676802948117,
'host_diversity': 0.7643010206520557}
{'model_seed': 2, 'method': 'r2_prototype_energy', 'stage': 1, 'current_angle':
-30.0, 'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 24, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.507557311999335, 'loss': 1.4733283966779709, 'current_ce':
1.3306295424699783, 'anchor_ce': 1.1631971821188927, 'balance':
0.026377016445621848, 'host_diversity': 0.5012058988213539}
{'model_seed': 2, 'method': 'r2_prototype_energy', 'stage': 2, 'current_angle':
0.0, 'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 24, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 3.3139688139999635, 'loss': 0.5711839664727449, 'current_ce':
0.43961181957274675, 'anchor_ce': 1.143204402178526, 'balance':
0.0069303122436394915, 'host_diversity': 0.3381037190556526}
{'model_seed': 2, 'method': 'r2_prototype_energy', 'stage': 3, 'current_angle':
60.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 24, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 3.9758223009994254, 'loss': 1.9931777194142342, 'current_ce':
1.866587571799755, 'anchor_ce': 1.1457834467291832, 'balance':
0.007031926972558722, 'host_diversity': 0.2332045892253518}
{'model_seed': 2, 'method': 'r2_prototype_energy', 'stage': 4, 'current_angle':
30.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]',
'context_checkpoint_hash':
'afd118c9727d830b09eda5acc9584cbdf1b657254a4d6c4512776d1d7f5fc73b',
'router_parameters': 24, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 4.001821404999646, 'loss': 0.8708494231104851, 'current_ce':
0.733026908710599, 'anchor_ce': 1.2732709720730782, 'balance':
0.009180361521430314, 'host_diversity': 0.2007280820980668}

```

===== MODEL SEED 3 =====

```

context {'model_seed': 3, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 1.4815483499996844, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.93125419318676, 'current_ce':
1.8885361105203629, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,
'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8518618531525135}
context {'model_seed': 3, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 2.485285378000299, 'forward_images': 2048,

```



```

'optimizer_steps': 16, 'loss': 1.5041144490242004, 'current_ce':
1.303204108029604, 'anchor_ce': 1.222976803779602, 'route_geo':
0.11352329840883613, 'context_geo': 0.03561043168883771, 'context_spread':
0.00016137384523062792, 'fiber': 0.5570544730871916, 'centroid':
0.015584335917992576, 'host_diversity': 0.7011856399476528}
context {'model_seed': 3, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 2.4864280269994197, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.6588918585330248, 'current_ce':
0.4661181913688779, 'anchor_ce': 1.182922426611185, 'route_geo':
0.1819728333503008, 'context_geo': 0.041285997023805976, 'context_spread':
1.0304579129982727e-05, 'fiber': 0.5085904225707054, 'centroid':
0.015085913975781295, 'host_diversity': 0.5194240286946297}
context {'model_seed': 3, 'stage': 3, 'current_angle': 60.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 3.4263983159999043,
'forward_images': 4096, 'optimizer_steps': 16, 'loss': 1.9793449118733406,
'current_ce': 1.7711356207728386, 'anchor_ce': 1.2790776640176773, 'route_geo':
0.25543148536235094, 'context_geo': 0.09215586120262742, 'context_spread':
6.457858034991659e-07, 'fiber': 0.529900262132287, 'centroid':
0.09696160990279168, 'host_diversity': 0.2839940916746855}
context {'model_seed': 3, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 4.317953458999909,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.9016415923833847,
'current_ce': 0.7023260463029146, 'anchor_ce': 1.2517721429467201, 'route_geo':
0.22530884761363268, 'context_geo': 0.07110582338646054, 'context_spread': 0.0,
'fiber': 0.5556021053344011, 'centroid': 0.07502247369848192, 'host_diversity':
0.2593129314482212}

```

--- r0_mlp ---

```

{'model_seed': 3, 'method': 'r0_mlp', 'stage': 0, 'current_angle': -60.0,
'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 4740, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 1.4976946510005291, 'loss': 1.9569604620337486, 'current_ce':
1.9135850220918655, 'anchor_ce': 0.0, 'balance': 0.002301847234775778,
'host_diversity': 0.865207128226757}
{'model_seed': 3, 'method': 'r0_mlp', 'stage': 1, 'current_angle': -30.0,
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 4740, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.5595733400004974, 'loss': 1.4907890781760216, 'current_ce':
1.328473974019289, 'anchor_ce': 1.2492046356201172, 'balance':
0.008587688731495291, 'host_diversity': 0.7393054589629173}
{'model_seed': 3, 'method': 'r0_mlp', 'stage': 2, 'current_angle': 0.0,
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 4740, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 3.3459767429994827, 'loss': 0.6931402869522572, 'current_ce':
0.5427357433363795, 'anchor_ce': 1.1883698292076588, 'balance':

```

```

0.02747338282642886, 'host_diversity': 0.6038777157664299}
{'model_seed': 3, 'method': 'r0_mlp', 'stage': 3, 'current_angle': 60.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 4740, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 3.291775874999985, 'loss': 2.027300201356411, 'current_ce':
1.8838765174150467, 'anchor_ce': 1.1926349811255932, 'balance':
0.046651074662804604, 'host_diversity': 0.4365531802177429}
{'model_seed': 3, 'method': 'r0_mlp', 'stage': 4, 'current_angle': 30.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 4740, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 4.037211894999928, 'loss': 0.8644140511751175, 'current_ce':
0.7149070855230093, 'anchor_ce': 1.2994828447699547, 'balance':
0.05190472467802465, 'host_diversity': 0.339268758893013}

--- r1_linear ---
{'model_seed': 3, 'method': 'r1_linear', 'stage': 0, 'current_angle': -60.0,
'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 20, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 1.3760718029998316, 'loss': 1.9519373998045921, 'current_ce':
1.9075073450803757, 'anchor_ce': 0.0, 'balance': 0.03292950487229973,
'host_diversity': 0.8556717038154602}
{'model_seed': 3, 'method': 'r1_linear', 'stage': 1, 'current_angle': -30.0,
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 20, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.568997076999949, 'loss': 1.47915867716074, 'current_ce':
1.318673562258482, 'anchor_ce': 1.242493413388729, 'balance':
0.024222232052125037, 'host_diversity': 0.7004930675029755}
{'model_seed': 3, 'method': 'r1_linear', 'stage': 2, 'current_angle': 0.0,
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 20, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 3.056492333999813, 'loss': 0.6819180008023977, 'current_ce':
0.5367112522944808, 'anchor_ce': 1.1766587905585766, 'balance':
0.017007596092298627, 'host_diversity': 0.5338097549974918}
{'model_seed': 3, 'method': 'r1_linear', 'stage': 3, 'current_angle': 60.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 20, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 3.2590631900002336, 'loss': 2.085949182510376, 'current_ce':
1.9496623426675797, 'anchor_ce': 1.171341560781002, 'balance':
0.01590489939553663, 'host_diversity': 0.3671486582607031}
{'model_seed': 3, 'method': 'r1_linear', 'stage': 4, 'current_angle': 30.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',

```

```

'router_parameters': 20, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 4.607461469000555, 'loss': 0.8362408839166164, 'current_ce':
0.6940574757754803, 'anchor_ce': 1.2572411820292473, 'balance':
0.014076515974011272, 'host_diversity': 0.31510939076542854}

--- r2_prototype_energy ---
{'model_seed': 3, 'method': 'r2_prototype_energy', 'stage': 0, 'current_angle':
-60.0, 'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 24, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 0.9735752149999826, 'loss': 1.8995160013437271, 'current_ce':
1.8553594127297401, 'anchor_ce': 0.0, 'balance': 0.08033958566375077,
'host_diversity': 0.8027921058237553}
{'model_seed': 3, 'method': 'r2_prototype_energy', 'stage': 1, 'current_angle':
-30.0, 'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 24, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.57264642500013, 'loss': 1.4011456593871117, 'current_ce':
1.253504417836666, 'anchor_ce': 1.1937474608421326, 'balance':
0.022813786868937314, 'host_diversity': 0.5425159987062216}
{'model_seed': 3, 'method': 'r2_prototype_energy', 'stage': 2, 'current_angle':
0.0, 'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 24, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.143251157999657, 'loss': 0.559478597715497, 'current_ce':
0.42441989947110415, 'anchor_ce': 1.1878544501960278, 'balance':
0.006001944946547155, 'host_diversity': 0.31946305837482214}
{'model_seed': 3, 'method': 'r2_prototype_energy', 'stage': 3, 'current_angle':
60.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 24, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 3.2675569419998283, 'loss': 2.103481002151966, 'current_ce':
1.9830402135849, 'anchor_ce': 1.081630952656269, 'balance': 0.00525596449006116,
'host_diversity': 0.2402978092432022}
{'model_seed': 3, 'method': 'r2_prototype_energy', 'stage': 4, 'current_angle':
30.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]',
'context_checkpoint_hash':
'e138aeac1dccf2dacf3052a2b3e417fe1b1f0e72dba61d128afa74b805801fef',
'router_parameters': 24, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 5.4814235620006, 'loss': 0.8300896473228931, 'current_ce':
0.699360191822052, 'anchor_ce': 1.1927437856793404, 'balance':
0.0051801976369461045, 'host_diversity': 0.22392113041132689}

===== MODEL SEED 4 =====
context {'model_seed': 4, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 0.9375051130000429, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.8844543397426605, 'current_ce':
1.841995544731617, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,

```

```

'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8466754965484142}
context {'model_seed': 4, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 1.582640395999988, 'forward_images': 2048,
'optimizer_steps': 16, 'loss': 1.5733205527067184, 'current_ce':
1.3762545697391033, 'anchor_ce': 1.1878099963068962, 'route_geo':
0.12663606088608503, 'context_geo': 0.04095319740008563, 'context_spread':
0.000171660228716064, 'fiber': 0.5041259415447712, 'centroid':
0.005447497329811313, 'host_diversity': 0.7207750156521797}
context {'model_seed': 4, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 2.1818134129998725, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.7092668786644936, 'current_ce':
0.511904938146472, 'anchor_ce': 1.1592403873801231, 'route_geo':
0.2413955256342888, 'context_geo': 0.04861066001467407, 'context_spread':
2.4153145290028988e-05, 'fiber': 0.45447569712996483, 'centroid':
0.020683884766185656, 'host_diversity': 0.5735614523291588}
context {'model_seed': 4, 'stage': 3, 'current_angle': 60.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 4.098451570000179,
'forward_images': 4096, 'optimizer_steps': 16, 'loss': 2.1761028692126274,
'current_ce': 1.9411439970135689, 'anchor_ce': 1.2091854326426983, 'route_geo':
0.43152554146945477, 'context_geo': 0.16377543797716498, 'context_spread':
9.244485186077345e-06, 'fiber': 0.4695909135043621, 'centroid':
0.2319379122927785, 'host_diversity': 0.3886672016233206}
context {'model_seed': 4, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 4.128389958000298,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.9243446514010429,
'current_ce': 0.7014490962028503, 'anchor_ce': 1.3020000085234642, 'route_geo':
0.31394029036164284, 'context_geo': 0.09928690921515226, 'context_spread':
5.3136723323632395e-06, 'fiber': 0.5687614642083645, 'centroid':
0.08600807655602694, 'host_diversity': 0.3726819735020399}

```

--- r0_mlp ---

```

{'model_seed': 4, 'method': 'r0_mlp', 'stage': 0, 'current_angle': -60.0,
'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'routier_parameters': 4740, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 0.9308287779995226, 'loss': 1.849764198064804, 'current_ce':
1.807383544743061, 'anchor_ce': 0.0, 'balance': 0.0009082660944841336,
'host_diversity': 0.8467051200568676}
{'model_seed': 4, 'method': 'r0_mlp', 'stage': 1, 'current_angle': -30.0,
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'routier_parameters': 4740, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.9686301640003876, 'loss': 1.4991547465324402, 'current_ce':
1.340503603219986, 'anchor_ce': 1.21938706189394, 'balance':
0.00932971631118562, 'host_diversity': 0.7249189540743828}
{'model_seed': 4, 'method': 'r0_mlp', 'stage': 2, 'current_angle': 0.0,
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':

```

```
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 4740, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.052650488000836, 'loss': 0.7002997156232595, 'current_ce':
0.5523183904588223, 'anchor_ce': 1.170560147613287, 'balance':
0.032443305593915284, 'host_diversity': 0.5860630683600903}
{'model_seed': 4, 'method': 'r0_mlp', 'stage': 3, 'current_angle': 60.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 4740, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 3.2835988070000894, 'loss': 2.030034326016903, 'current_ce':
1.8913149014115334, 'anchor_ce': 1.1447631381452084, 'balance':
0.04121238202787936, 'host_diversity': 0.4436497576534748}
{'model_seed': 4, 'method': 'r0_mlp', 'stage': 4, 'current_angle': 30.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 4740, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 3.231979008000053, 'loss': 0.84553562104702, 'current_ce':
0.697288915514946, 'anchor_ce': 1.2429357022047043, 'balance':
0.06290568690747023, 'host_diversity': 0.4161569196730852}
```

--- r1_linear ---

```
{'model_seed': 4, 'method': 'r1_linear', 'stage': 0, 'current_angle': -60.0,
'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 20, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 1.3904188350006734, 'loss': 1.8492202386260033, 'current_ce':
1.806872397661209, 'anchor_ce': 0.0, 'balance': 0.006110476067988202,
'host_diversity': 0.8408463597297668}
{'model_seed': 4, 'method': 'r1_linear', 'stage': 1, 'current_angle': -30.0,
'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 20, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.508869942000274, 'loss': 1.4970126822590828, 'current_ce':
1.3405929617583752, 'anchor_ce': 1.2124181538820267, 'balance':
0.007080585550284013, 'host_diversity': 0.6964772678911686}
{'model_seed': 4, 'method': 'r1_linear', 'stage': 2, 'current_angle': 0.0,
'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 20, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.180138259999694, 'loss': 0.6881656181067228, 'current_ce':
0.5453598890453577, 'anchor_ce': 1.1631414033472538, 'balance':
0.007853234594222158, 'host_diversity': 0.5219784900546074}
{'model_seed': 4, 'method': 'r1_linear', 'stage': 3, 'current_angle': 60.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 20, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 2.6252422500001558, 'loss': 2.0217952877283096, 'current_ce':
1.890301637351513, 'anchor_ce': 1.140573661774397, 'balance':
```

```

0.0060218907601665705, 'host_diversity': 0.34270436502993107}
{'model_seed': 4, 'method': 'r1_linear', 'stage': 4, 'current_angle': 30.0,
'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 20, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 4.153182345999994, 'loss': 0.8363858014345169, 'current_ce':
0.695001881569624, 'anchor_ce': 1.2450219094753265, 'balance':
0.006105742766521871, 'host_diversity': 0.3315285947173834}

--- r2_prototype_energy ---
{'model_seed': 4, 'method': 'r2_prototype_energy', 'stage': 0, 'current_angle':
-60.0, 'seen_angles': '[-60.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 24, 'forward_images': 1024, 'optimizer_steps': 16,
'wall_seconds': 0.9354045729996869, 'loss': 1.8169475123286247, 'current_ce':
1.7744549438357353, 'anchor_ce': 0.0, 'balance': 0.07882859138771892,
'host_diversity': 0.7710225805640221}
{'model_seed': 4, 'method': 'r2_prototype_energy', 'stage': 1, 'current_angle':
-30.0, 'seen_angles': '[-60.0, -30.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 24, 'forward_images': 2048, 'optimizer_steps': 16,
'wall_seconds': 1.5260962539996399, 'loss': 1.510609582066536, 'current_ce':
1.3576845414936543, 'anchor_ce': 1.2180069088935852, 'balance':
0.011441284033935517, 'host_diversity': 0.6110457628965378}
{'model_seed': 4, 'method': 'r2_prototype_energy', 'stage': 2, 'current_angle':
0.0, 'seen_angles': '[-60.0, -30.0, 0.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 24, 'forward_images': 3072, 'optimizer_steps': 16,
'wall_seconds': 2.077443721999771, 'loss': 0.6102391090244055, 'current_ce':
0.46193700283765793, 'anchor_ce': 1.262712199240923, 'balance':
0.012103442990337498, 'host_diversity': 0.4285141248255968}
{'model_seed': 4, 'method': 'r2_prototype_energy', 'stage': 3, 'current_angle':
60.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0]', 'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 24, 'forward_images': 4096, 'optimizer_steps': 16,
'wall_seconds': 2.6366119049998815, 'loss': 2.120865076780319, 'current_ce':
1.997305415570736, 'anchor_ce': 1.0611969530582428, 'balance':
0.004726065788418055, 'host_diversity': 0.3440732881426811}
{'model_seed': 4, 'method': 'r2_prototype_energy', 'stage': 4, 'current_angle':
30.0, 'seen_angles': '[-60.0, -30.0, 0.0, 60.0, 30.0]',
'context_checkpoint_hash':
'1a78273df8f0cd5b20db6be77d2749e52953fc601f3e1fbb795a9dd100ddffe3',
'router_parameters': 24, 'forward_images': 5120, 'optimizer_steps': 16,
'wall_seconds': 3.194875875999969, 'loss': 0.8822646476328373, 'current_ce':
0.7418468575924635, 'anchor_ce': 1.2396411299705505, 'balance':
0.006605964668779052, 'host_diversity': 0.32246782071888447}
Completed seeds: [0, 1, 2, 3, 4]

```

1.14 12. Seed aggregation and uncertainty

Source identity is the within-seed bootstrap unit. Model seed is the replication unit. Across seeds, the notebook reports paired means, Student-*t* 95% confidence intervals, standard deviation, and positive-seed fraction.

```
[17]: def concat_key(key):
        return pd.concat([run[key] for _, run in all_runs], ignore_index=True)

context_stage_df = concat_key("context_stage")
router_initialization_df = concat_key("router_initialization")
stage_df = concat_key("stage")
evaluation_df = concat_key("evaluation")
trace_df = concat_key("trace")
context_preservation_df = concat_key("context_preservation")
host_cost_df = concat_key("host_cost")
geometry_df = concat_key("geometry")
paired_geometry_df = concat_key("paired_geometry")
mediation_probe_df = concat_key("mediation_probe")
causal_raw_df = concat_key("causal_raw")
causal_summary_df = concat_key("causal_summary")
territory_df = concat_key("territory")
overlap_df = concat_key("overlap")
ablation_df = concat_key("ablation")
forgetting_df = concat_key("forgetting")
per_seed_method_df = concat_key("method_summary")

checkpoint_hashes = {str(seed): run["checkpoint_hash"] for seed, run in
    ↪all_runs}
initial_state_hashes = {str(seed): run["common_state_hash"] for seed, run in
    ↪all_runs}

compute_summary_df = stage_df.groupby(["model_seed", "method"], as_index=False).
    ↪agg(
        total_forward_images=("forward_images", "sum"),
        total_optimizer_steps=("optimizer_steps", "sum"),
        total_wall_seconds=("wall_seconds", "sum"),
    )

def seed_ci(values, confidence=0.95):
    values = np.asarray(values, dtype=float); n = len(values)
    mean = float(values.mean()); sd = float(values.std(ddof=1)) if n > 1 else 0.
    ↪0
    if n < 2: return mean, mean, mean, sd, n
    half = float(stats.t.ppf((1 + confidence) / 2, df=n - 1) * sd / math.
    ↪sqrt(n))
    return mean, mean - half, mean + half, sd, n
```

```

aggregate_method_rows = []
for method, block in per_seed_method_df.groupby("method"):
    for metric in ["nominal_route_rho", "functional_route_rho",
        ↪ "mean_heldout_accuracy", "final_trained_accuracy", "mean_forgetting"]:
        mean, low, high, sd, n = seed_ci(block[metric])
        aggregate_method_rows.append({
            "method": method, "metric": metric, "mean": mean,
            "seed_ci_low": low, "seed_ci_high": high, "seed_sd": sd,
        ↪ "seed_count": n,
            })
aggregate_method_df = pd.DataFrame(aggregate_method_rows)

aggregate_effect_rows = []
for keys, block in paired_geometry_df.groupby(["contrast", "partition",
        ↪ "geometry", "metric"]):
    mean, low, high, sd, n = seed_ci(block.delta_mean)
    favorable = block.delta_mean > 0 if keys[3] == "rho" else block.delta_mean
    ↪ 0
    aggregate_effect_rows.append({
        "contrast": keys[0], "partition": keys[1], "geometry": keys[2],
        ↪ "metric": keys[3],
        "mean_seed_effect": mean, "seed_ci_low": low, "seed_ci_high": high,
        "seed_sd": sd, "seed_count": n,
        "favorable_seed_fraction": float(np.mean(favorable)),
    })
aggregate_effect_df = pd.DataFrame(aggregate_effect_rows)

probe_seed_df = mediation_probe_df.pivot_table(
    index=["model_seed", "method"], columns="variant", values=["root_route_r2",
        ↪ "root_chord_error", "functional_transport_error"]
).reset_index()
probe_seed_df.columns = ["_".join([str(part) for part in col if str(part)]) if
    ↪ isinstance(col, tuple) else col for col in probe_seed_df.columns]

causal_seed_df = (
    causal_summary_df[causal_summary_df.scale != 0]
    .groupby(["model_seed", "method"], as_index=False)
    .agg(
        mean_nominal_csr=("nominal_csr", "mean"),
        mean_functional_csr=("functional_csr", "mean"),
        min_functional_csr_ci_low=("functional_csr_ci_low", "min"),
        mean_dmr=("dmr", "mean"),
        min_prediction_preservation_ci_low=("prediction_preservation_ci_low",
        ↪ "min"),
    )
)

```



```

)

ablation_seed_df = ablation_df.groupby(["model_seed", "method"],
↳as_index=False).agg(
    mean_top_host_drop=("accuracy_drop", lambda values: float(ablation_df.
↳loc[values.index].query("is_top_host == 1").accuracy_drop.mean())),
    mean_random_host_drop=("accuracy_drop", lambda values: float(ablation_df.
↳loc[values.index].query("is_random_control == 1").accuracy_drop.mean())),
)

aggregate_method_df.head(), aggregate_effect_df.head(), causal_seed_df.head()

```

```

[17]: (  method                metric      mean  seed_ci_low  seed_ci_high  \
0  r0_mlp      nominal_route_rho  0.311149    0.235937    0.386360
1  r0_mlp  functional_route_rho  0.309621    0.235330    0.383912
2  r0_mlp  mean_heldout_accuracy  0.569167    0.560628    0.577705
3  r0_mlp  final_trained_accuracy  0.630250    0.619851    0.640649
4  r0_mlp      mean_forgetting  0.097937    0.089704    0.106171

    seed_sd  seed_count
0  0.060573          5
1  0.059832          5
2  0.006877          5
3  0.008375          5
4  0.006631          5 ,

    contrast      partition      geometry  metric  mean_seed_effect  \
0  r1_linear_vs_r0_mlp  heldout_dense  functional    rho          0.032555
1  r1_linear_vs_r0_mlp  heldout_dense  functional  stress         -0.056849
2  r1_linear_vs_r0_mlp  heldout_dense    nominal    rho          0.051552
3  r1_linear_vs_r0_mlp  heldout_dense    nominal  stress         -0.036951
4  r1_linear_vs_r0_mlp      trained  functional    rho          0.068395

    seed_ci_low  seed_ci_high  seed_sd  seed_count  favorable_seed_fraction
0   -0.056460    0.121570  0.071690          5              0.6
1   -0.163770    0.050072  0.086111          5              0.8
2   -0.030279    0.133383  0.065904          5              0.6
3   -0.090689    0.016787  0.043279          5              0.8
4   -0.024158    0.160948  0.074540          5              0.8 ,

    model_seed      method  mean_nominal_csr  mean_functional_csr  \
0           0      r0_mlp          0.567897          0.156027
1           0      r1_linear          1.209845          1.593489
2           0  r2_prototype_energy          1.114813          1.144887
3           1      r0_mlp          0.622795          0.078106
4           1      r1_linear          1.513027          4.473651

    min_functional_csr_ci_low  mean_dmr  min_prediction_preservation_ci_low
0              0.005121  0.553340              1.000000

```

1	1.232366	1.214657	0.953125
2	0.956969	1.122701	0.921875
3	0.000162	0.598227	1.000000
4	2.106013	1.542533	0.976562)

1.15 13. Candidate gates and falsification logic

A positive result must pass mechanism-specific gates without hiding operational degradation.

- If R2 improves neither nominal nor functional route mediation, prototype routing fails to build the bridge.
- If functional geometry improves while nominal geometry does not, the old simplex metric was incomplete.
- If route mediation improves but host ablations remain factor-independent, the bridge stops at routing.
- If host effects improve while accuracy and retention remain neutral, v1.1.0 reaches functional but not operational geometry.
- If accuracy improves without mediation evidence, the system may be useful, but the geometric mechanism is not validated.

```
[18]: def get_effect(contrast, partition, geometry, metric):
    rows = aggregate_effect_df[
        (aggregate_effect_df.contrast == contrast)
        & (aggregate_effect_df.partition == partition)
        & (aggregate_effect_df.geometry == geometry)
        & (aggregate_effect_df.metric == metric)
    ]
    if rows.empty:
        raise KeyError((contrast, partition, geometry, metric))
    return rows.iloc[0]

primary_name = "r2_prototype_energy_vs_r0_mlp"
nominal_effect = get_effect(primary_name, "trained", "nominal", "rho")
functional_effect = get_effect(primary_name, "trained", "functional", "rho")
r2_causal = causal_seed_df[causal_seed_df.method == "r2_prototype_energy"]
r2_probe = mediation_probe_df[mediation_probe_df.method == "r2_prototype_energy"]
full_probe = r2_probe[r2_probe.variant == "full_context"]
controls = r2_probe[r2_probe.variant != "full_context"]
probe_pass_by_seed = []
for seed in MODEL_SEEDS:
    full = full_probe[full_probe.model_seed == seed].iloc[0]
    control = controls[controls.model_seed == seed]
    probe_pass_by_seed.append(
        full.root_route_r2 > control.root_route_r2.max()
        and full.functional_transport_error < control.
        functional_transport_error.min()
    )
```

```

r2_ablation = ablation_df[ablation_df.method == "r2_prototype_energy"]
mean_top_drop = r2_ablation[r2_ablation.is_top_host == 1].groupby("model_seed").
    ↳accuracy_drop.mean()
mean_random_drop = r2_ablation[r2_ablation.is_random_control == 1].
    ↳groupby("model_seed").accuracy_drop.mean()
host_ecology_positive = float(np.mean(mean_top_drop > mean_random_drop))

r2_perf = per_seed_method_df[per_seed_method_df.method ==
    ↳"r2_prototype_energy"].set_index("model_seed")
r0_perf = per_seed_method_df[per_seed_method_df.method == "r0_mlp"].
    ↳set_index("model_seed")
accuracy_delta = r2_perf.mean_heldout_accuracy - r0_perf.mean_heldout_accuracy
forgetting_delta = r2_perf.mean_forgetting - r0_perf.mean_forgetting
equivalence_margin =
    ↳float(protocol["candidate_gates"]["operational_equivalence_margin"])

candidate_gates = pd.DataFrame([
    {
        "gate": "C0_context_frozen_and_preserved",
        "passed": bool(context_preservation_df.max_abs_context_delta.max() <=
    ↳1e-6),
        "status": "protocol_integrity",
    },
    {
        "gate": "C0_equal_compute",
        "passed": bool(compute_summary_df.groupby("model_seed").apply(lambda b:
    ↳b.total_forward_images.nunique() == 1 and b.total_optimizer_steps.nunique()
    ↳== 1).all()),
        "status": "protocol_integrity",
    },
    {
        "gate": "C1_nominal_route_mediation",
        "passed": bool(nominal_effect.seed_ci_low > 0 and nominal_effect.
    ↳favorable_seed_fraction >= 0.80),
        "status": "candidate_only",
    },
    {
        "gate": "C1_functional_route_mediation",
        "passed": bool(functional_effect.seed_ci_low > 0 and functional_effect.
    ↳favorable_seed_fraction >= 0.80),
        "status": "candidate_only",
    },
    {
        "gate": "C1_probe_generalization",
        "passed": bool(np.mean(probe_pass_by_seed) >= 0.80),
    }
])

```

```

        "status": "candidate_only",
    },
    {
        "gate": "C4_functional_causal_specificity",
        "passed": bool((r2_causal.min_functional_csr_ci_low > 1.0).mean() >= 0.
↪80 and r2_causal.mean_functional_csr.mean() > 1.5),
        "status": "candidate_only",
    },
    {
        "gate": "C3_host_ecology",
        "passed": bool(host_ecology_positive >= 0.80),
        "status": "candidate_only",
    },
    {
        "gate": "C4_identity_preservation",
        "passed": bool((r2_causal.min_prediction_preservation_ci_low >= 0.95).
↪mean() >= 0.80),
        "status": "candidate_only",
    },
    {
        "gate": "C6_operational_non_degradation",
        "passed": bool((np.abs(accuracy_delta) <= equivalence_margin).mean() >=
↪0.80 and (forgetting_delta <= equivalence_margin).mean() >= 0.80),
        "status": "equivalence_not_superiority",
    },
    {"gate": "C5_memory_transport", "passed": False, "status":
↪"not_implemented"},
    {"gate": "operational_superiority", "passed": False, "status":
↪"not_preregistered_as_primary"},
])
candidate_gates

```

/tmp/ipykernel_17173/2433905066.py:47: DeprecationWarning:

DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```

"passed": bool(compute_summary_df.groupby("model_seed").apply(lambda b:
b.total_forward_images.nunique() == 1 and b.total_optimizer_steps.nunique() ==
1).all()),

```

```

[18]:

```

	gate	passed	status
0	C0_context_frozen_and_preserved	True	protocol_integrity
1	C0_equal_compute	True	protocol_integrity
2	C1_nominal_route_mediation	True	candidate_only
3	C1_functional_route_mediation	True	candidate_only

4	C1_probe_generalization	True	candidate_only
5	C4_functional_causal_specificity	False	candidate_only
6	C3_host_ecology	False	candidate_only
7	C4_identity_preservation	False	candidate_only
8	C6_operational_non_degradation	True	equivalence_not_superiority
9	C5_memory_transport	False	not_implemented
10	operational_superiority	False	not_preregistered_as_primary

1.16 14. Optional preregistered secondary mediation arm

This arm is disabled by default and must never be pooled with the architecture-only result. It asks a different question: can mediation be trained directly after the primary R2 result is reported?

A warm-up phase builds a frozen host-function cost matrix. The secondary margin loss is

$$\mathcal{L}_{\text{med}} = \text{ReLU}(m_{\text{med}} - d_{\text{func}}(r(c + \epsilon t), r(c)) + d_{\text{func}}(r(c + \epsilon v_{\perp}), r(c))).$$

The full training implementation is packaged as a callable helper, but execution requires `RUN_SECONDARY_MEDIATION_ARM=True` and exports to a separate method namespace.

```
[19]: def functional_mediation_margin_loss(router, contexts, tangent, orthogonal,
      ↪cost, scale=0.25, margin=0.05):
      tangent_local = local_tangent(contexts, tangent.expand_as(contexts))
      orthogonal_local = local_tangent(contexts, orthogonal.expand_as(contexts))
      base_route = router(contexts, temperature=TAU)
      tangent_route = router(F.normalize(contexts + scale * tangent_local,
      ↪dim=-1), temperature=TAU)
      orthogonal_route = router(F.normalize(contexts + scale * orthogonal_local,
      ↪dim=-1), temperature=TAU)
      tangent_change = entropic_transport_cost(
          tangent_route, base_route, cost,
          ↪
      ↪epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
          ↪
      ↪iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
      )
      orthogonal_change = entropic_transport_cost(
          orthogonal_route, base_route, cost,
          ↪
      ↪epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
          ↪
      ↪iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
      )
      return F.relu(float(margin) - tangent_change + orthogonal_change).mean()

if RUN_SECONDARY_MEDIATION_ARM:
```

```

    print("Secondary arm requested. Run it only after archiving the primary_
↪report; use a separate method and output directory.")
else:
    print("Secondary mediation arm: preregistered but disabled for the primary_
↪run.")

```

Secondary mediation arm: preregistered but disabled for the primary run.

1.17 15. Export immutable evidence

Every seed exports checkpoint hashes, initial-state hashes, compute, nominal and functional geometry, host costs, probes, causal results, host ecology, accuracy, NLL, forgetting, and claim status. The top-level manifest explicitly states that v1.1.0 is not yet a final C1–C6 qualification.

```

[20]: try:
    git_sha = subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).
↪strip()
except Exception:
    git_sha = "unknown"

root_out = Path("results/v1_1_0")
root_out.mkdir(parents=True, exist_ok=True)
for model_seed, run in all_runs:
    seed_out = root_out / f"seed_{model_seed}"
    seed_out.mkdir(parents=True, exist_ok=True)
    for filename, key in {
        "context_pretraining_stage_logs.csv": "context_stage",
        "router_initialization.csv": "router_initialization",
        "routing_stage_logs.csv": "stage",
        "staged_evaluation.csv": "evaluation",
        "dense_trace.pkl": "trace",
        "context_preservation.csv": "context_preservation",
        "host_cost_matrices.csv": "host_cost",
        "route_geometry.csv": "geometry",
        "paired_route_geometry_effects.csv": "paired_geometry",
        "mediation_probe_controls.csv": "mediation_probe",
        "causal_mediation_raw.csv": "causal_raw",
        "causal_mediation_summary.csv": "causal_summary",
        "host_territory.csv": "territory",
        "host_overlap_and_cost.csv": "overlap",
        "host_ablation.csv": "ablation",
        "forgetting.csv": "forgetting",
        "method_summary.csv": "method_summary",
    }.items():
        value = run[key]
        if filename.endswith(".pkl"):
            value.to_pickle(seed_out / filename)

```

```

        else:
            value.to_csv(seed_out / filename, index=False)
    (seed_out / "checkpoint_hashes.json").write_text(json.dumps({
        "context_checkpoint_hash": run["checkpoint_hash"],
        "common_host_classifier_initial_state_hash": run["common_state_hash"],
    }, indent=2), encoding="utf-8")

for filename, frame in {
    "context_pretraining_stage_logs.csv": context_stage_df,
    "router_initialization.csv": router_initialization_df,
    "routing_stage_logs.csv": stage_df,
    "staged_evaluation.csv": evaluation_df,
    "context_preservation.csv": context_preservation_df,
    "host_cost_matrices.csv": host_cost_df,
    "route_geometry.csv": geometry_df,
    "paired_route_geometry_effects.csv": paired_geometry_df,
    "aggregate_seed_effects.csv": aggregate_effect_df,
    "mediation_probe_controls.csv": mediation_probe_df,
    "causal_mediation_summary.csv": causal_summary_df,
    "causal_seed_summary.csv": causal_seed_df,
    "host_territory.csv": territory_df,
    "host_overlap_and_cost.csv": overlap_df,
    "host_ablation.csv": ablation_df,
    "forgetting.csv": forgetting_df,
    "per_seed_method_summary.csv": per_seed_method_df,
    "aggregate_method_summary.csv": aggregate_method_df,
    "compute_summary.csv": compute_summary_df,
    "gate_summary.csv": candidate_gates,
    "sensory_history.csv": pd.DataFrame(sensory_history),
}.items():
    frame.to_csv(root_out / filename, index=False)

claim_manifest = {
    "version": NOTEBOOK_VERSION,
    "status": "C0 complete implementation; empirical v1.1.0 claims pending_
↪ execution",
    "eligible_if_gates_pass": [
        "functional context-to-route mediation",
        "prototype-energy structural routing candidate",
        "host ecology candidate",
    ],
    "non_claims": protocol["non_claims"],
}
(root_out / "claim_manifest.json").write_text(json.dumps(claim_manifest,
↪ indent=2), encoding="utf-8")

manifest = {

```

```

    "notebook_version": NOTEBOOK_VERSION,
    "build_revision": BUILD_REVISION,
    "package_version": geometry_mmalls.__version__,
    "git_sha": git_sha,
    "run_profile": RUN_PROFILE,
    "model_seeds": MODEL_SEEDS,
    "context_checkpoint_hashes": checkpoint_hashes,
    "common_initial_state_hashes": initial_state_hashes,
    "split_manifest": split_manifest,
    "sensory_accuracy": sensory_accuracy,
    "methods": METHOD_SPECS,
    "primary_contrasts": PRIMARY_CONTRASTS,
    "functional_metric_evaluation_only": True,
    "secondary_mediation_arm_executed": RUN_SECONDARY_MEDIATION_ARM,
    "source_v109_results": "results/v1_0_9",
    "article": "docs/reports/
↳Geometry_MMALS_G1_v1_0_9_Results_and_v1_1_0_Specification.pdf",
    "reviewer_report": "docs/reports/
↳Geometry_MMALS_G1_Status_and_Perspective_Reviewer_Report_v1_2.pdf",
    "warning": "No final C1-C6 qualification may be claimed from CO
↳implementation alone.",
    "timestamp": time.time(),
}
(root_out / "run_manifest.json").write_text(json.dumps(manifest, indent=2),
↳encoding="utf-8")

import platform
(root_out / "environment.txt").write_text(
    "\n".join([
        f"python={sys.version}", f"platform={platform.platform()}",
        f"torch={torch.__version__}", f"device={DEVICE}",
    ]), encoding="utf-8"
)
print("Exported v1.1.0 evidence to", root_out.resolve())

```

Exported v1.1.0 evidence to /content/geometry-mmalls-g1/results/v1_1_0

1.18 16. Interpretation discipline and what remains after v1.1.0

1.18.1 A successful v1.1.0 result would mean

The ordered context is not merely a readable map: its geometry influences how work is distributed among hosts, and that influence survives source-disjoint probes and causal interventions.

1.18.2 It would still not prove

- operational superiority;
- mature host specialization;

- transport of memory through continual updates;
- bottom-up host feedback;
- full multi-objective energy control;
- RL or forward–backward control;
- distributed mycelial routing;
- quantum or quantum-inspired advantage.

1.18.3 Roadmap

- **G1.2 Host ecology:** heterogeneous roles, overlap, redundancy, ablation, recovery.
- **G1.3 Memory transport:** preserve or deliberately move geometric and functional relations through continual updates.
- **G2 Full Energy-Guided Router:** add host uncertainty, cost, memory risk, stability, and goal conditioning to E_h^{geo} .
- **G2.1 Bottom-up feedback:** hosts report confidence, novelty, saturation, capacity, and error.
- **G2.2 RL / predictive control:** optimize future consequences and reachable goals.
- **G3 Phase-aware / quantum-inspired routing:** amplitudes, phases, interference, and density matrices only after functional geometry is established.

The immediate scientific path is therefore understandable and falsifiable:

external factor → context geometry → route mediation → host function → operational value.

1.19 17. Qualification checklist

Before publication-level claims:

- ☐ run the five-seed pilot without manual source patching;
- ☐ verify exact context preservation and matched compute;
- ☐ archive every host cost matrix and partition hash;
- ☐ report R2–R0 and R2–R1, not only absolute R2 scores;
- ☐ keep nominal and functional route geometry separate;
- ☐ keep architecture-only and mediation-trained arms separate;
- ☐ report negative results and operational equivalence intervals;
- ☐ run ten seeds only after the functional mediation gate passes;
- ☐ update the reviewer report only from immutable exported CSVs.

```
[23]: import shutil
from pathlib import Path
import datetime
import os

# Define the source directory to be zipped
source_dir = REPO_DIR # This is '/content/geometry-mmalls-g1'

# Get the notebook filename from the 'notebook_path' variable
# This assumes 'notebook_path' was set in a previous cell, which it was.
# However, it might be None if the previous cell failed to find the path.
```

```

# Robustly get the current notebook path
notebook_id = os.environ.get('COLAB_JUPYTER_ITEM_ID')
if notebook_id:
    # Assuming notebook title is available in a variable named 'notebook_title'
    # or similar
    # For this specific notebook, we can infer it from the text cells.
    notebook_title = "Geometry_MMALS_G1_FunctionalRouting_v1_1_0" # Corrected
    # notebook title
    notebook_path = REPO_DIR / 'notebooks' / f'{notebook_title}.ipynb' # Added
    # 'notebooks' subdirectory
else:
    # Fallback for other environments or if ID is not available
    # Search for the .ipynb file in /content/ or known locations
    notebook_path_candidates = [
        REPO_DIR / 'notebooks' / 'Geometry_MMALS_G1_FunctionalRouting_v1_1_0.
        # ipynb',
        # Add other potential paths if known
    ]
    notebook_path = next((p for p in notebook_path_candidates if p.exists()),
    # None)

if notebook_path is None:
    # If still not found, raise an error or handle appropriately
    raise FileNotFoundError("Could not determine the current notebook's path.")

juniper_filename = Path(notebook_path).stem

# Get current timestamp
timestamp = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')

# Construct the new zip filename
zip_filename = f"{juniper_filename}_{timestamp}"
output_zip_path = Path('/tmp') / zip_filename

# Create the zip archive
shutil.make_archive(str(output_zip_path), 'zip', source_dir)

# Define the destination directory in Google Drive
drive_dest_dir = Path('/content/drive/MyDrive/MMALS')
drive_dest_dir.mkdir(parents=True, exist_ok=True)

# Define the final path for the zip file in Google Drive
final_zip_path = drive_dest_dir / f'{zip_filename}.zip'

# Move the zip file to Google Drive
shutil.move(f'{output_zip_path}.zip', final_zip_path)

```

```
print(f'"{zip_filename}.zip" has been successfully saved to: {final_zip_path}')
```

'Geometry_MMALS_G1_FunctionalRouting_v1_1_0_20260614_212819.zip' has been successfully saved to: /content/drive/MyDrive/MMALS/Geometry_MMALS_G1_FunctionalRouting_v1_1_0_20260614_212819.zip

```
[ ]: import os
```

```
!apt-get update
!apt-get install texlive texlive-xetex texlive-latex-extra pandoc
!pip install py pandoc
!pip -q install nbformat
```

```
0% [Working] Hit:1 http://archive.ubuntu.com/ubuntu jammy
InRelease
0% [Connecting to security.ubuntu.com] [Waiting for headers] [Connected to r2u.
Get:2 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
[3,632 B]
Get:3 https://cli.github.com/packages stable InRelease [3,917 B]
0% [Waiting for headers] [Connecting to security.ubuntu.com (104.20.28.246)] [2
0% [Waiting for headers] [Connecting to security.ubuntu.com (104.20.28.246)] [W
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
0% [4 InRelease 21.4 kB/128 kB 17%] [Waiting for headers] [Waiting for headers]
0% [Waiting for headers] [Waiting for headers] [Waiting for headers] [Connected
Get:5 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:7 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Get:8 https://cli.github.com/packages stable/main amd64 Packages [355 B]
Get:9 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
[18.1 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages
[1,611 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages
[7,489 kB]
Hit:12 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4,357
kB]
Get:14 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages [10.4 MB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages
[35.6 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages
[82.8 kB]
Get:17 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages
[4,006 kB]
Get:18 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages
[1,303 kB]
Get:19 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64
```

```

Packages [7,183 kB]
Get:20 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages [3,059 kB]
Get:21 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy/main amd64
Packages [38.6 kB]
Fetched 39.9 MB in 5s (8,855 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
pandoc set to manually installed.
The following additional packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
  fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
  libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
  libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
  libsynchronet2 libteckit0 libtexlua53 libtexlua53 libwoff1 libzip-0-13
  lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
  ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
  teckit tex-common tex-gyre texlive-base texlive-binaries
  texlive-fonts-recommended texlive-latex-base texlive-latex-recommended
  texlive-pictures texlive-plain-generic tipa xfonts-encodings xfonts-utils
Suggested packages:
  fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
  libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
  poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
  fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
  fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
  | postscript-viewer perl-tk xpdf | pdf-viewer xzdec
  texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
  icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
  texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
  texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
  default-jre-headless tipa-doc
The following NEW packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
  fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
  libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
  libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
  libsynchronet2 libteckit0 libtexlua53 libtexlua53 libwoff1 libzip-0-13
  lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
  ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
  teckit tex-common tex-gyre texlive texlive-base texlive-binaries
  texlive-fonts-recommended texlive-latex-base texlive-latex-extra
  texlive-latex-recommended texlive-pictures texlive-plain-generic

```

```

texlive-xetex tipa xfonts-encodings xfonts-utils
0 upgraded, 54 newly installed, 0 to remove and 30 not upgraded.
Need to get 182 MB of archives.
After this operation, 571 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-droid-fallback all
1:6.0.1r16-1.1build1 [1,805 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-lato all 2.0-2.1
[2,696 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 poppler-data all
0.4.11-1 [2,171 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-common all 6.17
[33.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-urw-base35 all
20200910-1 [6,367 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9-common
all 9.55.0~dfsg1-0ubuntu5.13 [753 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libidn12 amd64
1.38-4ubuntu1 [60.0 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/main amd64 libijs-0.35 amd64
0.35-15build2 [16.5 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjbig2dec0 amd64
0.19-3build2 [64.7 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9 amd64
9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libkpathsea6
amd64 2021.20210626.59705-1ubuntu0.3 [60.6 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy/main amd64 libwoff1 amd64
1.0.2-1build4 [45.2 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy/universe amd64 dvisvgm amd64
2.13.1-1 [1,221 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-lmodern all
2.004.5-6.1 [4,532 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-noto-mono all
20201225-1build1 [397 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-texgyre all
20180621-3.1 [10.2 MB]
Get:17 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libapache-pom-java
all 18-1 [4,720 B]
Get:18 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-parent-
java all 43-1 [10.8 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-logging-
java all 1.2-2 [60.3 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libptexenc1
amd64 2021.20210626.59705-1ubuntu0.3 [39.1 kB]
Get:21 http://archive.ubuntu.com/ubuntu jammy/main amd64 rubygems-integration
all 1.18 [5,336 B]
Get:22 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby3.0 amd64
3.0.2-7ubuntu2.12 [50.1 kB]

```

Get:23 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-rubygems all 3.3.5-2ubuntu1.2 [228 kB]
 Get:24 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby amd64 1:3.0~exp1 [5,100 B]
 Get:25 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rake all 13.0.6-2 [61.7 kB]
 Get:26 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-net-telnet all 0.1.1-2 [12.6 kB]
 Get:27 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-webrick all 1.7.0-3ubuntu0.2 [52.5 kB]
 Get:28 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-xmlrpc all 0.3.2-1ubuntu0.1 [24.9 kB]
 Get:29 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libruby3.0 amd64 3.0.2-7ubuntu2.12 [5,113 kB]
 Get:30 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libsyntax2 amd64 2021.20210626.59705-1ubuntu0.3 [55.8 kB]
 Get:31 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libteckit0 amd64 2.5.11+ds1-1 [421 kB]
 Get:32 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexlua53 amd64 2021.20210626.59705-1ubuntu0.3 [120 kB]
 Get:33 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexluajit2 amd64 2021.20210626.59705-1ubuntu0.3 [267 kB]
 Get:34 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libzip-0-13 amd64 0.13.72+dfsg.1-1.1 [27.0 kB]
 Get:35 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-encodings all 1:1.0.5-0ubuntu2 [578 kB]
 Get:36 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-utils amd64 1:7.7+6build2 [94.6 kB]
 Get:37 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 lmodern all 2.004.5-6.1 [9,471 kB]
 Get:38 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 preview-latex-style all 12.2-1ubuntu1 [185 kB]
 Get:39 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 t1utils amd64 1.41-4build2 [61.3 kB]
 Get:40 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 teckit amd64 2.5.11+ds1-1 [699 kB]
 Get:41 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-gyre all 20180621-3.1 [6,209 kB]
 Get:42 <http://archive.ubuntu.com/ubuntu> jammy-updates/universe amd64 texlive-binaries amd64 2021.20210626.59705-1ubuntu0.3 [9,861 kB]
 Get:43 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-base all 2021.20220204-1 [21.0 MB]
 Get:44 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-fonts-recommended all 2021.20220204-1 [4,972 kB]
 Get:45 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-latex-base all 2021.20220204-1 [1,128 kB]
 Get:46 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-latex-recommended all 2021.20220204-1 [14.4 MB]

```

Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive all
2021.20220204-1 [14.3 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all
1:1.8.16-2 [207 kB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all
1:1.8.16-2 [5,199 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures
all 2021.20220204-1 [8,720 kB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:54 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 3s (56.8 MB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 118243 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...

```

```

Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package rubygems-integration.
Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.

```



```

Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libtexluajit2:amd64.
Preparing to unpack
.../32-libtexluajit2_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libzip-0-13:amd64.
Preparing to unpack .../33-libzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.
Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...
Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package t1utils.
Preparing to unpack .../38-t1utils_1.41-4build2_amd64.deb ...
Unpacking t1utils (1.41-4build2) ...
Selecting previously unselected package teckit.

```

```

Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../41-texlive-
binaries_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.
Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack .../45-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive.
Preparing to unpack .../46-texlive_2021.20220204-1_all.deb ...
Unpacking texlive (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.
Preparing to unpack .../47-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...
Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../48-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../49-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../50-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...
Selecting previously unselected package texlive-plain-generic.
Preparing to unpack .../51-texlive-plain-generic_2021.20220204-1_all.deb ...
Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../52-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.
Preparing to unpack .../53-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...

```

```
[ ]: # Explicitly setting the notebook filename for this execution,
# as there isn't a robust automatic detection method in Colab.
notebook_filename = "Geometry_MMALS_G1_FunctionalRouting_v1_1_0.ipynb"
notebook_name_without_ext = os.path.splitext(notebook_filename)[0]

# Define the full path to the notebook in Google Drive
full_notebook_path_drive = f"/content/drive/MyDrive/Colab Notebooks/
↳{notebook_filename}"
local_notebook_path = f"/content/{notebook_filename}"

# Copy the notebook to the local /content/ directory to avoid path issues with
↳nbconvert
print(f"Executing: cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\"")
!bash -c "cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\""
```

```
[ ]: !jupyter nbconvert \
--to pdf \
--output-dir='/content' \
--PDFExporter.latex_command=["'xelatex', '{filename}',
↳'-interaction=nonstopmode'"] \
"/content/Geometry_MMALS_G1_FunctionalRouting_v1_1_0.ipynb"
```